

Agents in the Large: Perception-Centered Architecture for Persistent Agents

Shihan Dou* Haoxiang Jia* Shichun Liu* Feng Chen Chenhao Huang Yujiong Shen
 Shaofan Liu Jiayi Chen Jiahang Lin Honglin Guo Qianyu He Minghao Guo Ziyi Ye
 Pluto Zhou Tao Gui Qi Zhang Xuanjing Huang

NLP Lab, Fudan University PL Lab, Peking University
 CCDS, Nanyang Technological University Hunyuan Team, Tencent

Abstract

Cognitive language agents have achieved substantial progress by equipping language models with memory, tools, and decision-making procedures, enabling agents to reason and act in interactive environments. Existing frameworks largely cast these agents as systems for solving user-specified, bounded tasks. An increasingly important goal is for language agents to provide persistent assistance in long-lived settings where user needs, context, and service procedures persist and change, and to remain useful across the broad range of tasks that arise over time. Yet we still lack a framework to characterize persistent AI agents, organize existing work, and guide future development. To this end, we propose a **Perception-Centered Architecture for Persistent Agents (Pera)**. Pera describes a persistent agent organized around perception and control components that continually perceive service-relevant signals from episodic task executions, internal context, and changes in the surrounding environment, and use these signals to construct lifecycle tasks. These tasks drive the ongoing operation and adaptation of the agent’s service procedures. We use Pera to retrospectively organize recent work, examine a detailed case study, and offer forward-looking insights for building more capable persistent agents. Just as software engineering moved from programming in the small to programming in the large, Pera frames the evolution of language agents as an analogous architectural transition toward long-lived, adaptive intelligence systems.

1 Introduction

Software development once underwent a major transition from *programming in the small* to *programming in the large* (DeRemer & Kron, 1976). As software came to support larger systems over longer periods of operation (Lehman, 1980), the focus of design expanded from correctly implementing individual programs (Parnas, 1972; 1971) to sustaining systems that provide reliable service over time. Software architecture (Perry & Wolf, 1992; Garlan et al., 1993) came to address the organization of system components, interactions, and the constraints and rationale required to satisfy system-level requirements. In parallel, research on software evolution and dependability treated continuing change, maintenance, fault tolerance, recovery, and reliable service as lifecycle concerns (Patterson et al., 2002; Garlan et al., 2004; De Lemos et al., 2013; Li et al., 2024). Thus, the object of design expanded from a bounded program execution to a persistent system that must remain reliable as requirements and operating conditions change (Oreizy et al., 1998).

Language agents are now approaching an analogous architectural transition (Fan & Lan, 2026). As their capabilities and operational horizons expand, they are increasingly expected to become persistent agents (Fang et al., 2025; Gao et al., 2025; Zheng et al., 2026a) that provide persistent assistance (Li et al., 2025a) in

*Equal contribution. Each author may list their name first. Contact: shihandou@foxmail.com, haoxiangjia@stu.pku.edu.cn, plutozhou096@foxmail.com, and tgui@fudan.edu.cn.

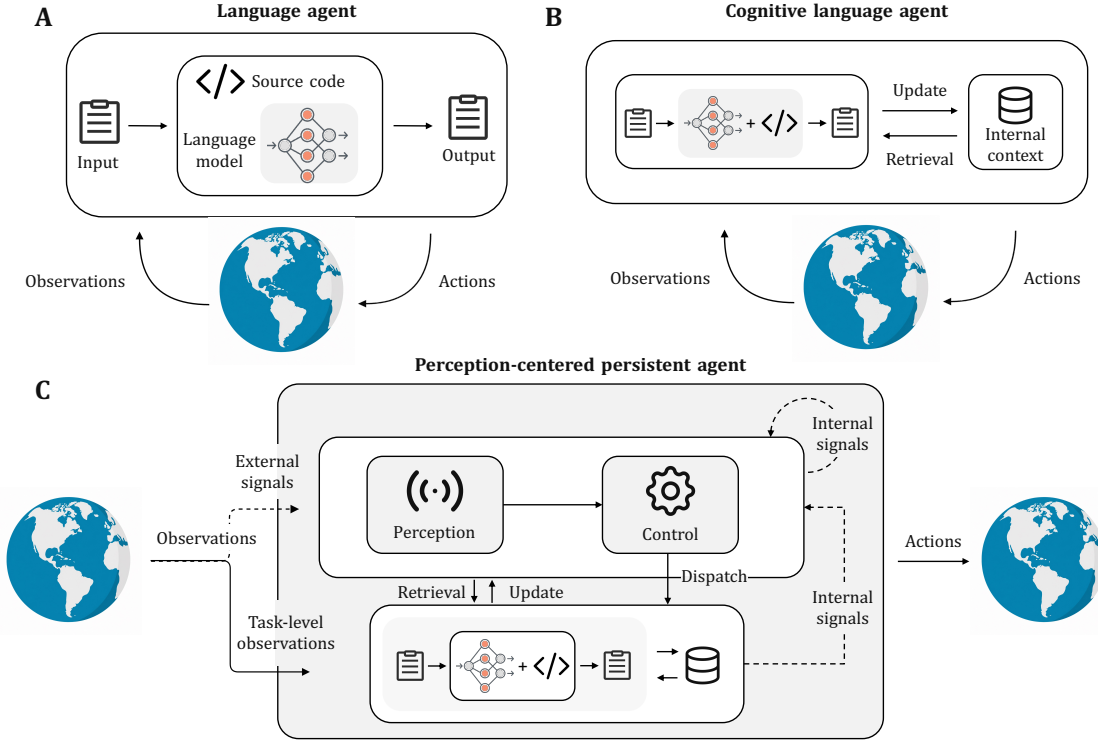


Figure 1: Comparison of three language agents. **A**: Language agents place a language model within an agent decision procedure implemented in source code, and interact with the external environment through grounding. **B**: Cognitive language agents augment this interaction loop with internal context (i.e., memory) and cognitive actions such as reasoning, retrieval, and updating, allowing retained experience to inform subsequent tasks. **C**: To sustain effective service in a long-lived setting, the agent must continually perceive changes in itself and its environment and proactively adapt accordingly. Perception-centered persistent agents introduce active perception and lifecycle-level control to notice such changes and proactively update the agent and its service.

long-lived settings (Xu et al., 2026c; Zhu et al., 2026), beyond the completion of any single user-specified task. However, most language agents are still architected as cognitive systems (Sumers et al., 2023; Wang et al., 2024a; Yao et al., 2022b) that coordinate language models, memory, tools, and decision-making procedures around a current user task (Yao et al., 2022b; Shinn et al., 2023; Schick et al., 2023; Yu et al., 2026; Du et al., 2026). Although recent agent systems have been run continuously for tens of hours while attempting a single complex task, empirical evaluations show that their effective task horizons remain much shorter and that progress often plateaus during extended runs (Starace et al., 2025; Kwa et al., 2026; Xu et al., 2026a). This degree of autonomy still falls short of persistent agency.

Consider computer-use agents, one of the most actively studied classes of language agents. Current evaluations of computer-use agents focus on completing open-ended tasks in real or simulated computer environments (Sager et al., 2026; Wang et al., 2026b). A more autonomous computer-use agent should do more than complete tasks explicitly assigned by the user (Nushi et al., 2019; Lu et al., 2025; Li et al., 2026a). It should continuously perceive the user’s activities (Yang et al., 2025c; Tang et al., 2026a) and changes across the computer environment (Pasternak et al., 2025; Li et al., 2026a) and infer needs that have not yet been articulated (Lu et al., 2025; Lan et al., 2026). It should also recognize when its current decision procedure (Cox, 2005; Hu et al., 2025a; Lin et al., 2026) is inadequate for anticipated future tasks and revise it accordingly (Zhang et al., 2024b; Du et al., 2026; Zhang et al., 2026a). In other words, persistent assistance requires an agent not only to decide how to address the task at hand, but also to infer what additional work is needed and assess whether its current procedures remain adequate.

This example illustrates a broader transition from cognitive language agents, or *agents in the small*, to persistent agents, or *agents in the large* (Figure 1). The focus of agent design expands from solving a user-specified task to providing persistent assistance throughout the lifetime of a long-lived setting. A user-specified task provides an explicit objective, but a long-lived setting presents signals whose relevance to future service must be actively recognized. It also exposes far more information than a resource-bounded agent can process continuously. Therefore, *active perception* is key to this transition (Bajcsy, 1988; Bajcsy et al., 2018; Lu et al., 2025). Active perception forms the interface through which an evolving, open-ended setting becomes observable to lifecycle-level control. It allows the agent to anticipate emerging needs and prepare for changes without waiting for explicit instructions. What is missing is an architectural account of persistent agents that specifies which components must exist beyond the cognitive core and how perceived change becomes work the agent performs on itself.

To this end, we propose **Pera**, the **P**erception-Centered **A**rchitecture for **P**ersistent **A**gents, a conceptual framework that organizes the core components and operating principles of persistent agents. Pera augments cognitive language agents with *perception* and *control* components that support persistent operation across the lifecycle of a setting. It distinguishes episodic tasks, which complete specific pieces of work, from lifecycle tasks, which sustain and improve the agent’s continuing service of a setting. The perception component continually senses and processes service-relevant signals from both external and internal sources. External signals arise from changes in the environment, while internal signals arise from task execution and the agent’s internal context. The control component interprets these signals and formulates lifecycle tasks. Lifecycle tasks can inspect and update the agent’s context, revise its decision procedures, or anticipate emerging user needs and prepare relevant support before they are articulated as explicit requests.

We further use Pera to retrospectively organize recent work and clarify how existing efforts contribute to persistent agents, and prospectively suggest actionable directions for building future persistent agents.

Paper organization. The remainder of the paper is organized as follows. We first review a historical shift in software system design and early work that incorporated perception into agent architectures in Section 2. We explain why perception is essential for agents serving long-lived settings in Section 3. Then, Section 4 introduces the Pera framework and uses it to organize representative prior work. Section 5 further illustrates Pera through a detailed case study, and Section 6 offers several actionable insights for advancing persistent agents. Finally, Section 7 discusses related agent frameworks, and Section 8 concludes the present paper.

2 Background

Three architectural ideas provide the basis for our account of persistent agents. Programming in the small and programming in the large distinguish computation within individual units from control over relations among them (DeRemer & Kron, 1976). Cognitive architectures for language agents organize the machinery for executing bounded tasks (Sumers et al., 2023). Active perception shows that information acquisition is itself guided by an agent’s activities and information needs (Hayes-Roth, 1995). Together, these ideas allow us to ask what must change architecturally when an agent’s service persists beyond any single task.

2.1 Programming in the small and programming in the large

A software module can correctly implement its local function while the larger system remains incorrect because system properties can depend on relations among modules (Ockerbloom & Allen, 1995; Garlan et al., 1995). Individually correct modules can rely on incompatible assumptions, expose mismatched interfaces, or interact in ways that violate constraints enforced by none of them. Such properties cannot be addressed solely through computation internal to an individual module.

Programming in the small concerns computation within an individual program or module, whereas programming in the large concerns the organization of relations among such units (DeRemer & Kron, 1976). When a property depends on interfaces, dependencies, or interactions that span multiple units, the architecture must explicitly organize those relations. Therefore, programming in the large does not make the computation inside each unit larger; it adds control over properties that no individual unit organizes. These two levels are complementary: programming in the large organizes relations among units implemented in the small

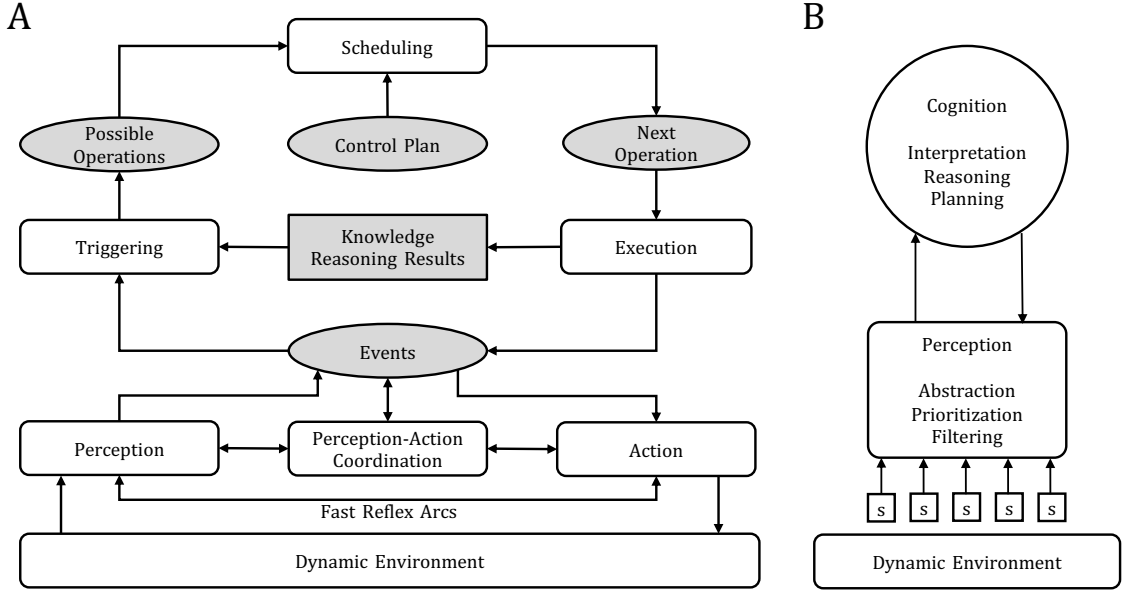


Figure 2: Active perception in adaptive intelligent systems (AIS), adapted from Hayes-Roth (Hayes-Roth, 1995). **A:** AIS coordinates perception, reasoning, and action through adaptive control. **B:** Perception selectively acquires environmental information under cognitive guidance, forming a feedback loop between information acquisition and control.

without replacing their internal computation. Architectural scale is determined by the scope of activity and state placed under control, rather than by the amount of computation performed within a unit. We use this criterion to distinguish architectural scales in language agents.

2.2 Language agents as agents in the small

Cognitive architectures for language agents place a language model within a system of memory, actions, and decision-making procedures that together support task execution (Sumers et al., 2023; Fan & Lan, 2026). During execution, observations update the task state, while the decision procedure uses reasoning, retrieval, and interaction with the environment to select subsequent actions. In task-oriented language agents, this machinery is typically organized around a bounded objective, that directs execution, determines which information is relevant, and defines when the task is complete. We call tasks organized around such an objective an *episodic task*, where *episodic* refers to the scope of the tasks rather than a type of memory.

We call an architecture an *agent in the small* when the episodic task is the highest-level unit that it explicitly organizes. An episodic task may involve extensive planning, tool use, interaction, testing, and revision, and could extend across many actions or hours of execution (Ma et al., 2024; Starace et al., 2025; Kwa et al., 2026). The agent could also retain long-term memory, learn reusable procedures, and reuse experience across tasks. Neither extended execution nor persistent state enlarges the architectural scope so long as they remain subordinate to the active episodic objective. Architectural scope expands only when persistent conditions and relations across tasks themselves become objects of control.

2.3 Active perception in adaptive intelligent systems

Task-directed control also determines what information an agent needs to acquire. A dynamic environment exposes more information than a resource-bounded agent can process continuously, so perception must select among available sources according to the agent’s current activities and information needs (Bajcsy, 1988; Bajcsy et al., 2018).

Hayes-Roth’s architecture for adaptive intelligent systems places this selection under adaptive control (Hayes-Roth, 1995). As illustrated in Figure 2, cognition guides which environmental information enters subsequent reasoning and at what level of detail. Guardian provides a concrete example: as a patient’s condition, incoming data, computational resources, and reasoning demands change, the system changes what information it inspects rather than applying a fixed perceptual strategy (Hayes-Roth et al., 1992). Therefore, perception is not merely a fixed input stage, but part of the agent’s controlled operation.

The agent’s current activities create information needs that guide perception, while the resulting observations update its assessment of the situation and thereby change subsequent reasoning and perceptual choices. We use *active perception* to refer to this feedback relation between information acquisition and control. What is sensed, when it is sensed, and at what level of detail can change with the activities and conditions being served.

Therefore, information relevance is relative to the activity that perception serves. Within a bounded task, the task objective determines the agent’s current information needs. More generally, active perception couples information acquisition to the scope of activity under control: as that scope changes, what needs to be sensed can change with it.

3 From agents in the small to agents in the large

Persistent agency changes the unit of control. When service persists beyond an episodic task, no active task objective fully specifies what must be perceived, maintained, or adapted for future service. Therefore, a persistent architecture must organize a scope that persists across tasks, make changes to that scope observable, and convert relevant changes into bounded tasks that can affect future service. These requirements lead respectively to the notions of a *long-lived setting*, active perception across task boundaries, and *lifecycle tasks*.

3.1 Episodic tasks within long-lived settings

An episodic task ends with its bounded objective, but many of the conditions under which tasks are interpreted and performed persist beyond that objective (Zhong et al., 2024; Wu et al., 2024a; Yu et al., 2026). Users retain preferences, artifacts and decisions persist, and rules and procedures continue to affect later tasks (Tan et al., 2025; Rezazadeh et al., 2025; Li et al., 2025c; Zhao et al., 2024; Zhang et al., 2026b). We call the persistent scope formed by the users, artifacts, constraints, decisions, and procedures whose effects survive individual tasks a *long-lived setting*. Episodic tasks belong to the same setting when persistent conditions produced, assumed, or modified in one task can affect the interpretation or execution of another (Maharana et al., 2024; Ong et al., 2025; Bian et al., 2026). The long-lived setting is the scope of continuing service rather than a particular memory representation; an agent’s retained context represents only the parts of that setting it has captured.

A long-lived setting is distinct from a long-horizon task. A long-horizon task extends execution under one objective and ends when that objective is resolved (Kwa et al., 2026; Zhang et al., 2026f). A long-lived setting instead spans multiple objectives and persists beyond the completion of any one of them. Thus, *long-horizon* describes the extent of execution within a task, whereas *long-lived* describes the scope of coordination across tasks (Zheng et al., 2025a).

Cross-task persistence means that completing an episodic task does not resolve all conditions relevant to later tasks. Earlier decisions constrain later tasks, delayed feedback can revise prior assumptions, and artifacts, policies, tools, user needs, and reusable procedures can change over time (Shen et al., 2026; Zhang et al., 2026e; Patel, 2026; Chen et al., 2026c). Therefore, previously valid context can become stale, and procedures that once succeeded can cease to fit current conditions. These changes can affect later tasks even when the current episodic objective has been successfully completed.

Cognitive language agents provide the machinery to respond once such conditions enter an active task. An *agent in the large* adds a higher level of organization that treats the long-lived setting itself as the scope over which persistent conditions and cross-task relations are maintained (Zhu et al., 2026). As in programming in the large, task-level cognition retains its own computation while a broader architecture controls properties

that span individual tasks. Therefore, an agent in the large is not simply a longer-running task agent; it explicitly organizes conditions that persist beyond any individual task.

3.2 Active perception makes the setting observable

Within an episodic task, the objective creates the agent’s information needs. It directs the agent toward relevant artifacts, memories, and environmental states, and observations are evaluated according to whether they support task completion (Hayes-Roth, 1995). Therefore, a bounded objective provides a criterion for what information matters during task execution.

However, across task boundaries, information can remain relevant after the objective that first exposed it has ended. A change matters at the level of the long-lived setting when it can alter persistent context, constraints, procedures, or user needs on which later tasks depend. A correction to a completed result can revise an assumption that matters to later tasks (Shinn et al., 2023; Patel, 2026); a procedure can become unreliable after its operating environment changes; and new conditions relevant to later tasks can arise before, between, or after episodic tasks (Li et al., 2026a; Yang et al., 2025c). If information enters the agent only through the observation loop of an active task, the agent cannot respond to such changes until their consequences re-enter through a later task. Therefore, task-scoped perception leaves an observability gap across task boundaries. This gap concerns what the architecture makes available for control, not how much past information it is capable of storing.

Active perception closes this gap by extending information acquisition from the current task to persistent conditions shared across tasks (Hayes-Roth, 1995; Bajcsy et al., 2018). It inspects outcomes and feedback from completed tasks, persistent context and reusable procedures, and changes in the surrounding setting (Zhang et al., 2026d; Li et al., 2026b). Because the relevance of these sources changes as the setting evolves, sensing must adapt to what the agent currently needs to know. The agent’s current understanding of the setting guides subsequent sensing, while newly perceived information updates that understanding.

The significance of a change can also emerge only across time. A single failed action does not establish that a reusable procedure is obsolete, whereas repeated failures following an interface change provide evidence of a persistent mismatch (Zhu et al., 2026). By relating evidence across tasks, active perception makes persistent changes observable at the level where they affect later tasks.

Perception is necessary but insufficient for persistent adaptation. The same evidence may indicate a local exception, a persistent change, or a condition outside the agent’s authority (Maes, 1994; Horvitz, 1999; Rhodes & Maes, 2000). Perception makes such conditions available for judgment, but does not determine whether or how the agent should respond. That requires a level of control that evaluates the perceived condition and turns selected changes into bounded interventions.

3.3 Lifecycle tasks make perceived changes actionable

A perceived condition affects later tasks only when the architecture can act on its implications. Such tasks may assess a suspected persistent change, update stale context, revise an unreliable procedure, verify that an adaptation has the intended effect, or prepare for an emerging recurring need (Zhang et al., 2024b; Sun et al., 2026a). We call bounded tasks whose primary purpose is to maintain or improve conditions shared by future tasks a *lifecycle task*. By contrast, an episodic task pursues a bounded user-facing outcome. The distinction follows from both purpose and completion criterion: an episodic task completes with respect to its bounded user-facing outcome, whereas a lifecycle task completes with respect to the evidence or change required to assess, establish, restore, or prepare a persistent condition on which later tasks depend.

Proactivity alone does not make a task lifecycle-level. An agent-initiated task remains episodic when its purpose is to produce a one-off user-facing outcome, while a user can directly request a lifecycle task when the requested change is intended to govern later tasks. For example, preparing one report is episodic, whereas correcting a preference that should govern future reports is a lifecycle task (Tan et al., 2025). A single request can also contain both kinds of tasks: a request to prepare a report and use the same format for future reports induces an episodic task for the immediate report and a lifecycle task for the persistent formatting

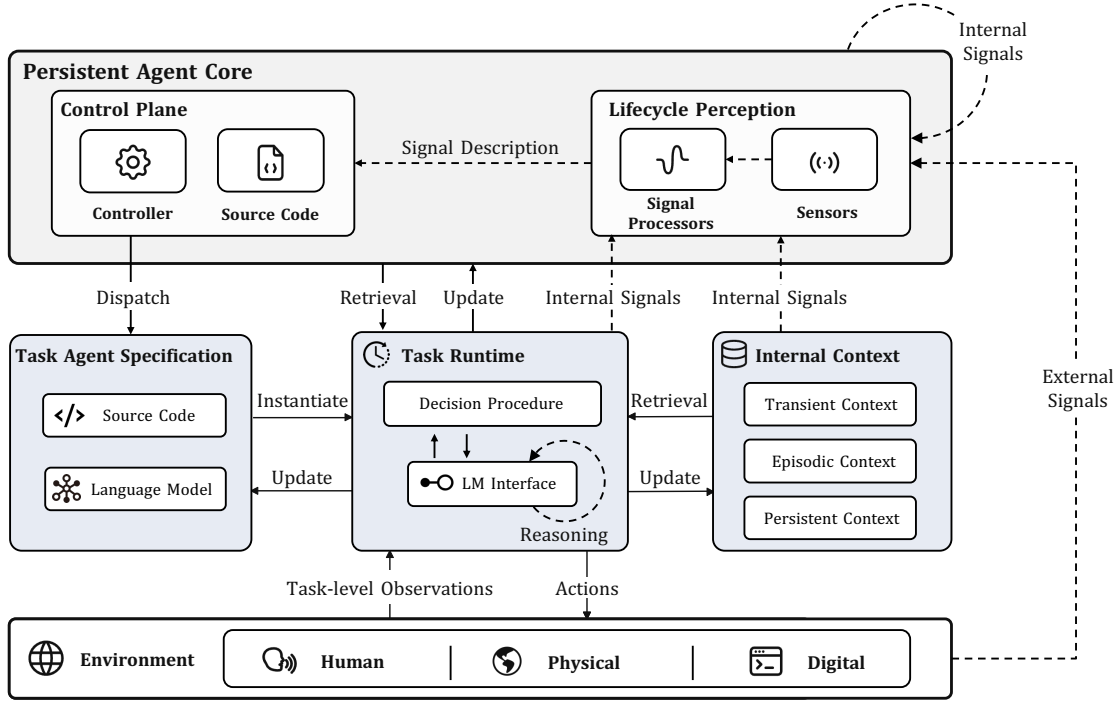


Figure 3: The conceptual perception-centered architecture for persistent agents (**Pera**). Pera couples active perception with task-oriented cognition, using perceived signals to formulate lifecycle tasks that update the agent and its service as the long-lived setting evolves. Signals can be external or internal: external signals come from human, physical, and digital environments, whereas internal signals come from a single task execution, reviews across multiple task executions and internal context memory, or perception and control themselves.

preference (Wu et al., 2026a). Therefore, what matters is not who initiates the tasks or whether the objectives appear in the same interaction, but the purpose of each task and what its completion is intended to establish.

Lifecycle tasks reuse the same memory, reasoning, planning, and action machinery that executes episodic tasks (Sumers et al., 2023; Shinn et al., 2023). Therefore, the architectural change is not a new mechanism for bounded execution, but a new scope of control over why such a task is formulated and what its completion is meant to establish. Episodic execution advances the current user-facing objective, whereas lifecycle execution produces evidence or changes intended to maintain conditions shared by later tasks.

Active perception and lifecycle tasks together add a level of organization above episodic task execution. Perception makes cross-task changes observable, lifecycle control determines which changes require intervention, and task-level cognition carries out the resulting bounded tasks. This is the architectural transition from agents in the small, whose highest-level unit of control is the episodic task, to agents in the large, whose control extends to the continuing service of a long-lived setting.

4 Pera: A Perception-Centered Architecture for Persistent Agents

We present Pera as an architecture for extending cognitive language agents (Laird et al., 1987; Sumers et al., 2023; Fan & Lan, 2026) from episodic task execution to continuing service in long-lived settings. As shown in Figure 3, Pera introduces a **persistent agent core** that couples lifecycle perception with a control plane. **Lifecycle perception** captures external signals from the environment and internal signals from task execution, internal context, and the agent’s own components, and interprets them to produce descriptions of service-relevant changes. The **control plane** evaluates these changes and determines whether they require further evidence or work. When adaptation is needed, it formulates a **lifecycle task package** and dispatches

it to the task agent specification, which instantiates a task runtime to carry out the work through task-level decision-making. The **task agent specification**, together with the lifecycle task package, instantiates a **task runtime** with a decision procedure specialized to the task. Through task-level decision-making, the runtime uses language-model reasoning to perform internal actions or interact with the external environment.

Pera organizes this work into two types of bounded tasks. An episodic task pursues a concrete user-facing outcome, whereas a lifecycle task establishes or restores a persistent condition on which future service depends. A task’s type is determined by its purpose and completion condition rather than by whether the user or agent initiates it. Both are executed through the same task-level machinery, but lifecycle tasks additionally return evidence for lifecycle-level review, which determines whether their persistent effects should be accepted, further revised, or rolled back.

Therefore, Pera forms a recurring loop between perception, control, and task-oriented cognition: perception makes changes in the long-lived setting observable; control turns relevant changes into executable lifecycle work; task execution and review establish and assess persistent effects; and their outcomes generate further signals as the setting and agent continue to evolve. The following sections describe lifecycle perception and signals (Section 4.1), lifecycle task packages (Section 4.2), the shared action space (Section 4.3), task-level decision-making (Section 4.4), and lifecycle-level decision-making (Section 4.5).

4.1 Signals and Perception

A *signal* indicates an event or state change that may be relevant to future service. The lifecycle perception captures and interprets such signals for lifecycle-level judgment, enabling proactive responses to relevant changes.

Signals and observations differ in the architectural scope of their use. An *observation* is information supplied to a task runtime during execution to inform its subsequent decisions Nowé et al. (2012); Shinn et al. (2023); Ding et al. (2026). For example, a user message or feedback returned by a grounding action can serve as an observation (Huang et al., 2023a; Gou et al., 2025; Wu et al., 2026b). By contrast, a signal makes a change available to the persistent agent core as the starting point of lifecycle-level judgment. Signals arise during a task, after a task has ended, and when no task is active. The same event can serve both roles. For instance, a tool error supports recovery within the current runtime while also providing evidence that a reusable procedure has become unreliable Lin et al. (2026); Li (2025).

According to where the indicated change occurs, signals can be divided into external and internal signals.

External signals. External signals indicate changes in the setting surrounding the agent. Their sources parallel the physical, human, and digital environments with which cognitive language agents interact through grounding actions (Sumers et al., 2023). Physical signals can come from cameras, microphones, or sensors that measure conditions such as temperature and motion (Yoon et al., 2026; Chai et al., 2018; Wang et al., 2025e). Human signals reflect changes in user activity such as feedback on prior work or new needs Kaufmann et al. (2023). Digital signals indicate changes in software states and artifacts, such as file-system events or updated access policies Hong et al. (2024b); Nguyen et al. (2025); Wang et al. (2026d).

Recent proactive agents implement parts of this external perceptual interface Liao et al. (2023); Zhang et al. (2024c). ProactiveAgent monitors user activities and environmental events to predict when assistance is useful (Lu et al., 2025). ContextAgent and ProAgent use open-world sensory signals, including video and audio captured by wearable devices, to infer user context and emerging needs (Yang et al., 2025b;c). These systems show how events outside an episodic task can trigger agent assistance.

Internal signals. Internal signals indicate changes exposed by task executions or by the agent’s internal context and components. For example, execution traces expose recurring tool failures, while reviews expose source-code changes that fail to produce their intended effects (Liu et al., 2026a; Chen et al., 2026a). Runtime monitoring methods already inspect task trajectories to detect risks and execution anomalies (Dou et al., 2026a; Luo et al., 2025; Gehring et al., 2024; Dou et al., 2024).

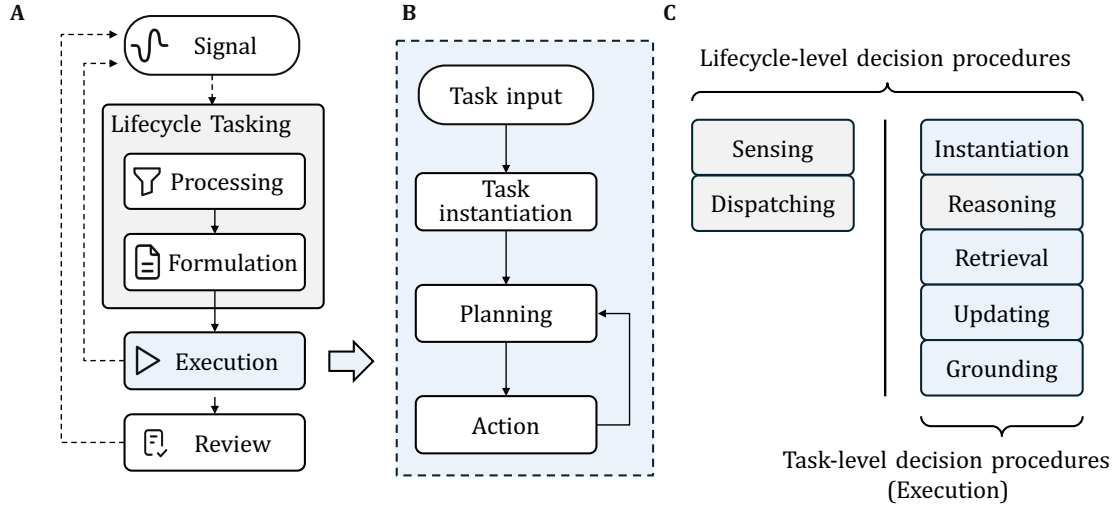


Figure 4: Lifecycle-level and task-level decision procedures and the action space in Pera. **A:** Lifecycle-level decision-making actively senses and interprets signals, determines whether to formulate lifecycle tasks, executes them through task-level decision-making, and reviews their execution and outcomes. Processing and formulation can invoke task-level execution when their work requires it, with the resulting outputs or signals feeding back into the ongoing lifecycle-level decision-making process. Execution and review can in turn generate new signals. **B:** Task-level execution begins with a task input, either a task prompt or a lifecycle task package. The agent instantiates a task runtime using the task agent specification and, when available, the task-specific procedure in the package, and then runs a repeated planning–action cycle until completion or failure. **C:** Sensing and dispatching are specific to lifecycle-level decision procedures, while reasoning can support both lifecycle-level signal processing and formulation and task-level planning. During task-level decision-making, planning selects internal actions, including retrieval and updating, or external grounding actions. Their outcomes can in turn become signals captured through sensing.

Changes in internal context can likewise generate signals. The lifecycle perception and scheduled tasks inspect how retained context evolves and identify changes relevant to continuing service. For example, accumulated information that reveals an emerging user interest triggers proactive assistance, whereas recurring stale entries expose deficiencies in context maintenance and can lead to a lifecycle task that revises the relevant procedure or source code (Chao et al., 2026; Xu et al., 2026b; Zhang et al., 2025c; Yang et al., 2026a).

Signals represent either individual events or conditions inferred from related events Cugola & Margara (2012). For example, the creation of a directory can produce a single-event signal. A single failed tool call does not establish a persistent problem Chen et al. (2025). Repeated failures following an interface change instead provide signals that a reusable procedure has become obsolete. Signal aggregation allows the lifecycle perception to recognize conditions that are difficult to identify from one event alone (Wang et al., 2025b; Chowa et al., 2026).

Lifecycle perception. The lifecycle perception comprises two components for capturing and interpreting signals. Specifically, sensors detect signals, while signal processors interpret them to produce descriptions of what has happened and make these descriptions available to the control plane.

Sensors can take physical or software forms. Physical sensors capture measurements or multimodal inputs from the surrounding environment (Driess et al., 2023; Reed et al., 2022; Mu et al., 2023). Software sensors can receive events from operating systems and applications or monitor the agent’s internal state and activity (Agashe et al., 2025; Wu et al., 2024b; Xie et al., 2024). Sensors capture signals either as events occur or through periodic inspection of relevant state (Ziafati et al., 2013; Rivkin et al., 2023). The resulting inputs can describe an individual event or summarize changes observed over an interval.

Signal processors interpret the captured signals and describe the indicated condition with sufficient specificity for subsequent judgment. Signal processing suppresses irrelevant inputs and combines related signals when a condition becomes evident only over time. A sequence of temperature signals, for example, can be interpreted as indicating that the workspace temperature has remained above a specified threshold (Rivkin et al., 2024; 2023; Gyllstrom et al., 2006). Signals collected from several task executions can similarly be summarized as a recurring interface failure. For changes that require multimodal understanding, retrieval, or tool use, signal interpretation can itself be packaged as a lifecycle task and executed by a task runtime (Majumdar et al., 2024).

The lifecycle perception passes the resulting description to the control plane, which evaluates the described change against the current states of the agent and its environment and determines whether further evidence or a lifecycle task is required. If the description is incomplete, further sensing or verification can itself be packaged as a task. Section 4.5 describes this subsequent process.

Although existing work has explored parts of this perceptual process, external and internal sources can produce many potentially irrelevant signals, making appropriate choices about sensing intensity and granularity critical Zaky et al. (2020). Accurately interpreting signals and identifying their underlying causes remain a core challenge for lifecycle perception. We revisit these challenges and their design implications in Section 6.

4.2 Lifecycle Task Packages

By analyzing an interpreted signal that describes what has happened, the control plane determines what work should follow and constructs a *lifecycle task package*. The resulting package converts the required work into a form that can be dispatched and executed. As the interface between the control plane and task-level execution, it specifies the task objective and procedure, together with operational requirements such as context acquisition, failure recovery, and review Hong et al. (2024a); Gao et al. (2024a).

At the center of the package is a clear task objective describing what the runtime should accomplish and what persistent effect should remain after execution. The package also defines a completion criterion that specifies the evidence the runtime must produce before it stops and returns its result. For example, suppose a computer-use agent repeatedly fails to submit expense reports after the reporting website changes its interface Yan et al. (2025); Ishmam & Marino (2026). In this example, the package instructs the runtime to inspect the updated interface and revise the reusable submission procedure. Its completion criterion can require the runtime to return the revised procedure together with a trace showing successful submission of a test expense report through the updated interface.

To make the objective executable, the package can also provide or reference a procedure that guides execution. The procedure can reuse an existing workflow, introduce task-specific code, or equip the runtime with a new tool for grounding in the environment (Wang et al., 2023a; Nguyen et al., 2024; Cai et al., 2024). The package also supplies the context needed for execution or specifies how it should be retrieved. In the example above, it can provide the failed task traces and the current submission procedure, then instruct the runtime to inspect the changed interface before proposing a revision.

Moreover, lifecycle tasks that modify reusable procedures or source code can affect later services when the update fails Moll et al. (2026); Ren et al. (2026). Therefore, for tasks that modify persistent components, the package includes recovery mechanisms that preserve a stable version and restore it when the proposed change fails (Li et al., 2025b; Hosek & Cadar, 2013; Giuffrida et al., 2013). In the running example, the runtime retains the previous submission procedure and restores it when the revision fails validation. The package can also restrict modification to specified components or require additional authorization before consequential actions are executed (Wang et al., 2025a; Zhang & Zhang, 2026; South et al., 2025; Zhu et al., 2025).

The runtime determines task completion, whereas the lifecycle level determines whether the returned change becomes part of future service. Therefore, the package further specifies a review criterion stating what must hold before a proposed change is accepted (Zhang et al., 2026a; Moll et al., 2026). Review extends beyond completion: it examines evidence that the runtime was not required to produce, in particular whether the change preserves behavior the agent already supported. In the running example, completion requires one successful submission through the updated interface, whereas review additionally replays the revised

procedure on the report types that succeeded before the change, and only then does the revision replace the stored procedure Lou et al. (2024). The assessment work can be performed within the original execution procedure, or dispatched as a separate task when an independent evaluator is needed (Singhi et al., 2026; Zhuge et al., 2024b; Chan et al., 2024). The acceptance decision itself remains with the control plane.

In practice, these elements do not prescribe a fixed package structure. The control plane adapts the package’s contents and level of detail to the difficulty and expected impact of each lifecycle task. If lifecycle tasks repeatedly expose deficiencies in package construction, revising the package-formulation procedure can itself become a lifecycle task Zhang et al. (2025b); Hu et al. (2025a).

4.3 Action Space

Pera defines an action space covering operations within the agent and between the agent and the external environment. It comprises *sensing*, *dispatching*, *instantiation*, *reasoning*, *retrieval*, *updating*, and *grounding*, as shown in Figure 4. Reasoning, retrieval, updating, and grounding build on cognitive language agents’ internal and external actions (Sumers et al., 2023; Wang et al., 2024a). Pera further incorporates sensing, dispatching, and instantiation for ongoing perception and lifecycle-level coordination.

Sensing. Sensing proactively monitors external and internal sources for signals relevant to continuing service. Depending on signal source and monitoring objective, sensing can respond to individual events or periodically inspect a relevant state. For external signals, ProactiveAgent (Lu et al., 2025) monitors user activities and environmental events; ContextAgent (Yang et al., 2025c) and ProactiveVA (Zhao et al., 2025) infer user context and emerging needs from multimodal or interface observations. For internal signals, runtime monitors inspect task trajectories to detect unsafe states, anomalous steps, and policy violations (Wang et al., 2025b; Liu et al., 2026a).

Sensing mechanisms also differ in how much computation they use and how signals are collected over time. Sensing can adaptively allocate perception, using lightweight observations for routine monitoring while invoking costlier perception when additional evidence is needed (Ren et al., 2024; Zaky et al., 2020). Moreover, long-running sensing requires sustained observation, as relevant conditions can emerge only across events or state changes at different moments (Maldaner et al., 2026; Ziafati et al., 2013).

While recent work has explored different signal sources and sensing mechanisms, how sensing should adapt as the environment and agent state change remains understudied in recent language agents. Persistent agents need to expand or revise the sources they monitor, adjust sensing frequency and granularity, and determine when related events should be considered together (Ren et al., 2024).

Dispatching. Dispatching sends a formulated lifecycle task package from the control plane to the agent specification and determines when the package enters execution. The control plane chooses among immediate dispatch, delayed execution, and waiting for resources, dependencies, or authorization Yu et al. (2025a); Guo et al. (2026a). At the system level, AIOS (Mei et al., 2024) schedules concurrent agent requests and manages their access to shared model and tool resources. For dependent-subtask workflows, DynTaskMAS (Yu et al., 2025a) uses dynamic task graphs to support asynchronous and parallel execution.

Dispatching can account for resource availability, task dependencies, task priority, and execution state Xie et al. (2025); Zheng et al. (2026b). Agent.xpu (Wei et al., 2025) maintains separate reactive and proactive workload priority queues, and Agent libOS (Zhang, 2026) supports scheduling, authorization, and resumption of long-running agents. For persistent agents, dispatching must also consider how lifecycle tasks that maintain or improve future service should be scheduled alongside episodic tasks with immediate response requirements.

Instantiation. Instantiation turns the reusable agent specification into an executable task runtime for a particular task. The action binds the task input to the general decision procedure. The agent specification encodes this procedure in the agent’s source code and language-model parameters (Sumers et al., 2023; Gao et al., 2024a; Wu et al., 2023b). A common implementation makes this transition implicit: a task instruction directly initiates the agent’s decision procedure (Yao et al., 2022b; Huang et al., 2022; Singh et al., 2022). Pera explicitly models this transition as an instantiation action, analogous to creating a running instance

from a reusable program. For a lifecycle task, instantiation applies the package’s task-specific procedure to supplement or constrain the general decision procedure before execution.

Reasoning. Reasoning allows a task runtime to process the information currently available to generate new information. Unlike retrieval, which brings stored information into the runtime, reasoning transforms the information already available (Sumers et al., 2023; Wei et al., 2022; Yao et al., 2023; Hao et al., 2023). It can interpret the most recent observation and revise its plan (Wang et al., 2023a; Skreta et al., 2024), distill insights from execution trajectories (Zhao et al., 2024; Ouyang et al., 2026), or synthesize information retrieved from internal context (Park et al., 2023; Lee et al., 2024). It can also identify additional context to retrieve, evaluate which grounding action to take, and guide updates to internal context or agent components Hao et al. (2023). In Pera, reasoning can also support sensing and signal processing by helping the signal processor interpret captured signals or analyze the resulting signal description to clarify what it indicates and inform the formulation of a subsequent lifecycle task package Zeng et al. (2022). Active-perception agents similarly use reasoning or reflection to determine when and where to acquire observations (Zhang et al., 2024a; Lee et al., 2025; Wang et al., 2026e). When reasoning reveals that necessary evidence is unavailable because the current sensing procedure is insufficient, the task execution can produce an internal signal that prompts the control plane to formulate a lifecycle task for revising the relevant sensing or signal-processing procedure.

Retrieval. Retrieval reads relevant information from internal context for decision-making Park et al. (2023); Packer et al. (2023); Wang et al. (2024c). Pera organizes internal context into transient, episodic, and persistent context. Transient context is task-local information temporarily stored outside the language model’s context window and retrieved as needed during the same execution. Episodic context retains knowledge and experience accumulated through task executions for later use, while persistent context retains information relevant to continuing service in the long-lived setting. Depending on context representation, retrieval uses rule-based filtering, sparse lexical matching such as BM25 (Robertson & Zaragoza, 2009), dense embedding retrieval (Karpukhin et al., 2020), or neural ranking and late interaction (Nogueira & Cho, 2019; Khattab & Zaharia, 2020).

Recent agent-memory systems retrieve different forms of accumulated context (Hu et al., 2025b), including salient information extracted from multi-session interactions in Mem0 (Chhikara et al., 2025), records organized as dynamically linked notes in A-MEM (Xu et al., 2026b), and textual strategies distilled from successful and failed experiences in ReasoningBank (Ouyang et al., 2026). In Pera, retrieval is limited to internal context, since the general decision procedure is applied through instantiation, while information from the external environment is acquired through grounding.

Updating. Updating changes the agent’s internal state or implementation. Language agents can update internal context by retaining task experience, consolidating knowledge, and revising accumulated information (Zhong et al., 2024; Li et al., 2025c; Ouyang et al., 2026). At the agent-specification level, model parameters can be updated through diverse learning methods (Zeng et al., 2024; Sun et al., 2025; Hu et al., 2026); source-code elements and reusable procedures can be added, revised, or removed to improve task-level decision-making (Hu et al., 2025a; Shang et al., 2025; Ni et al., 2026).

Viewed through Pera, recent self-evolving agents mainly update the source code, tools, and reusable procedures in the agent specification (Lee et al., 2026; Lin et al., 2026; Zhang et al., 2026c). For example, STOP (Zelikman et al., 2023) improves an LM-based decision procedure encoded in source code; SICA (Robeyns et al., 2025) and other recent work (Yin et al., 2025; Zhang et al., 2026c) modify agent source code; and some agent frameworks (Zhang et al., 2026a; Liu et al., 2026b) extend updating to broader execution-harness components.

In Pera, updating can also modify the persistent agent core. For example, a deficiency in sensors or the controller can motivate a lifecycle task that revises the affected component itself. Updating can directly revise software sensors and signal processors (Murphy & Hershberger, 1999; Peters & Knoll, 2024). Changes to physical sensors are carried out through grounding: a task runtime can use embodied action to recalibrate, repair, or replace a sensor, or proactively ask the user to make such a change. Such changes can alter the physical sources from which the lifecycle perception captures future signals Murphy & Hershberger (1999); Peters & Knoll (2024). Updating can then revise the corresponding software interface or grounding procedure

to incorporate a changed or newly introduced device. Like updates to model parameters and source code, updates to the persistent agent core are risky because errors can affect perception (Shao et al., 2026a).

Grounding. Grounding allows a task runtime to interact with the external environment through available mechanisms, such as software tools, communication channels, and embodied action (Zhou et al., 2024; Qin et al., 2024; Shao et al., 2026b). Feedback from this interaction is returned to the runtime as an observation for subsequent decisions. Unlike sensing, which captures service-relevant changes, grounding is performed by a task runtime to advance its current task.

Recent work has explored how environmental interaction is incorporated into agent decision-making (Yao et al., 2022a; Zhang et al., 2025a). Existing language-agent frameworks also consider digital environments, interactions with humans, and physical environments as broad forms of external grounding (Sumers et al., 2023; Zhou et al., 2024).

In Pera, grounding interactions across these environments can reveal information about the user or long-lived setting that matters beyond the current task. For example, user edits can reveal latent preference patterns, while direct feedback can indicate a new requirement or a setting change (Sun et al., 2026a; Gao et al., 2024b; Shao et al., 2026b). The lifecycle perception can also capture information produced or revealed during grounding as an external signal, allowing the control plane to determine whether to formulate a lifecycle task that updates service procedures. Recent agents have begun learning latent preferences from user edits and behavioral histories (Gao et al., 2024b; Wang et al., 2026c) and adapting to user desires across embodied or repeated interactions (Wang et al., 2025d; Mehri et al., 2026).

Sections 4.4 and 4.5 describe how these actions are organized within task-level and lifecycle-level decision-making, respectively.

4.4 Task-Level Decision-Making

Task-level decision-making is the process through which a task runtime carries out an individual task. A task enters task-level execution either directly through a task request or through a dispatched lifecycle task package. As illustrated in Figure 4, it first enters instantiation, after which the resulting runtime proceeds through a repeated cycle of planning and action De Silva et al. (2020); Sumers et al. (2023); Huang et al. (2023b); Yao et al. (2022b); Georgeff & Ingrand (1989). For work dispatched through the control plane, the task-level decision-making process realizes the execution stage.

Task instantiation. Task instantiation applies the task input to the agent specification to create a task runtime Shoham (1993); Laird et al. (1987). For a directly provided task request, it initializes the general decision procedure defined by the agent specification with that request Sumers et al. (2023). When the input is a lifecycle task package, its task-specific procedure and operational requirements further supplement or constrain the general procedure. Notably, language-model computation occurs during execution of this instantiated procedure, while the LM interface is depicted separately in Figure 3 to make explicit how the procedure exchanges prompts and responses with the language model.

Planning. Planning produces and selects the next action given the task objective and the runtime’s current task state Yao et al. (2023); Hao et al. (2023); Zhou et al. (2023). Within planning, the instantiated decision procedure uses reasoning actions to generate candidate actions and assess their suitability or anticipated consequences before selecting what to execute next Yao et al. (2022b); Huang et al. (2023b). If additional information is needed, planning selects retrieval or grounding as the next action. The result of that action can then inform a later planning cycle.

Recent agent frameworks differ in how candidate actions are generated, whether and how they are evaluated, and how far the procedure looks ahead before selecting one Yao et al. (2023); Wang et al. (2023b). For example, Soar’s (Laird et al., 1987) decision cycle proposes and evaluates operators before one is selected for application, while CoALA (Sumers et al., 2023) describes language-agent planning in terms of proposing, evaluating, and selecting candidate actions. ReAct (Yao et al., 2022b) produces the next action incrementally

from the evolving interaction history, whereas SayCan (Ahn et al., 2022) selects among predefined robotic skills by combining language-model predictions with estimated feasibility.

More recent work makes the comparison of alternatives before execution increasingly explicit. Tree of Thoughts (Yao et al., 2023) generates and evaluates multiple candidate reasoning states, while RAP (Hao et al., 2023) and LATS (Zhou et al., 2023) use search and value estimates to compare longer reasoning or action trajectories before selection. A complementary line of work (Song et al., 2022; Dagan et al., 2023; Munoz-Avila et al., 2025) imposes explicit structure on planning, for example by revising plans using execution feedback, integrating language models with symbolic planners, or organizing actions into hierarchical or behavior-tree representations.

Action. The action stage executes the retrieval, grounding, or updating operation selected during planning Schick et al. (2023); Wang et al. (2024b). The outcome is returned to the runtime as an action result and contributes to its current task state for the next planning cycle.

Language agents represent and execute selected actions in different ways Patil et al. (2024). For example, Toolformer (Schick et al., 2023) and Gorilla (Patil et al., 2024) encode external API calls in model outputs, whereas CodeAct (Wang et al., 2024b) represents actions as executable code interpreted by a code runtime. Some agent frameworks (Qin et al., 2025; Feng et al., 2026) interleave language-model reasoning with tool execution and incorporate returned results into subsequent decisions, while others translate model decisions into direct interface operations such as keyboard and mouse actions. Planned steps can also be delegated to specialized execution components (Lu et al., 2023; Patil et al., 2024; Agashe et al., 2025). Execution outcomes can then be compared with their expected effects to detect failures and guide subsequent recovery (Zhang et al., 2026h).

Planning and action repeat until the task reaches its completion condition or the runtime can no longer continue under the current procedure and constraints. A failed action result can trigger a retry or a revised course of action in a later cycle. The runtime returns task results and artifacts on completion, or available evidence of failure when it cannot continue. When these outputs or execution traces reveal a service-relevant condition, the lifecycle perception can capture the evidence as an internal signal.

4.5 Lifecycle-Level Decision-Making

Lifecycle-level decision-making is the process through which Pera interprets external and internal signals and organizes the work needed to sustain and improve service in a long-lived setting Kephart & Chess (2003); Garlan et al. (2004); Park et al. (2023). As illustrated in Figure 4, it proceeds through four recurring stages, i.e., *processing*, *formulation*, *execution*, and *review*. *Processing* transforms an incoming signal into a description of the perceived condition, expressed in a domain-specific language that the control component can reason over. Based on this interpretation, *formulation* defines lifecycle tasks and constructs the corresponding task packages described in Section 4.2, supplying the content needed for their *execution* and *review*.

When existing procedures are insufficient, formulation generates task-specific procedure code and includes it in the package. Each package is dispatched to a task agent specification and applied during instantiation, where the task-specific procedure in the package supplements or constrains the general decision procedure defined by the specification. This creates a task runtime with a decision procedure specialized to the lifecycle task. The runtime executes the task through task-level decision-making and returns results for review against the applicable requirements. Execution and review can produce further signals, allowing the process to continue as the setting or the agent changes.

Processing. Signal processors aggregate the signals acquired by the sensors described in Section 4.1 and transform them into descriptions of perceived conditions, expressed in a domain-specific language. Different systems instantiate this interface according to their sensing domains. For example, recent wearable and proactive-agent systems (Yu et al., 2025b; Yang et al., 2025c; Zhang et al., 2026g) translate continuous multimodal sensor streams into textual descriptions or align the streams with textual descriptions and contexts for language-based reasoning. Embodied systems (Driess et al., 2023; Gu et al., 2024; Gao et al., 2026) similarly incorporate visual, proprioceptive, thermal, and other sensor modalities into language models

or summarize multisensory observations as semantic scene representations and natural-language explanations. The resulting description is passed to formulation.

Formulation. Formulation starts from the interpretation produced by processing and determines whether lifecycle work should follow and, if so, how it should be organized. The control plane uses a language model as its general reasoning mechanism and plays a role analogous to an operating-system core, handling basic coordination directly while delegating extended work to task runtimes through lifecycle tasks. For each task, it constructs the corresponding package with the elements described in Section 4.2. Among these elements, the task-specific procedure is the one produced during formulation itself, and it supplements or constrains the general decision procedure defined by the task agent specification during instantiation.

The task-specific procedure can be coded directly by the control plane, as illustrated by agents that dynamically construct executable actions or tools when predefined capabilities are insufficient (Sur’is et al., 2023; Liang et al., 2023; Qian et al., 2023). Alternatively, its construction or validation can be formulated as another lifecycle task when direct coding exceeds the control plane’s computational capacity or requires extended work Hong et al. (2024a); Hu et al. (2025a). Moreover, when the available signals are insufficient or their interpretation is inadequate, the control plane can also formulate lifecycle tasks to extend or revise the relevant sensing and processing components, such as by adapting sensor configurations or dynamically orchestrating modality-specific perception tools (Tao et al., 2025; Chen et al., 2026e; Wu et al., 2023a). Once formulation is complete, the control plane dispatches each package to a task agent specification for instantiation and execution.

Execution. Both episodic and lifecycle tasks are carried out through the task-level decision-making process described in Section 4.4. To maintain Pera’s architectural abstraction, a lifecycle task originating from a user request is also formulated by the control plane as a lifecycle task package and is then dispatched for execution. The execution process can produce new internal signals, such as execution errors. These signals provide evidence for formulating lifecycle tasks that revise deficient components, such as revising source code based on failures (Zhang et al., 2026a; Shinn et al., 2023; Zhuge et al., 2024a). Additionally, execution results can produce internal signals, such as indications that an outcome fails to meet expectations Yang et al. (2023); Guo et al. (2024); Gou et al. (2024). These signals are examined and consolidated in the subsequent review stage and are also captured by the lifecycle perception.

Review. In the review phase, the agent examines execution-related evidence, such as the execution trajectory, task outcome, and resulting state, to surface internal signals Madaan et al. (2023); Zhuge et al. (2024b); Shinn et al. (2023). Review remains a distinct lifecycle-level stage, although parts of the review process are carried out through additional task-level execution.

For an episodic task, review evaluates the immediate execution, such as whether the task objective was achieved, whether the result is correct, and whether the resulting state is consistent with the user’s request (Huang et al., 2026a; Zhang et al., 2026h). For a lifecycle task, the task-specific procedure in the package can further specialize this review. Beyond what is examined for an episodic task, review of a lifecycle task also determines whether the intended persistent condition was established and whether any resulting update should be accepted, further revised, or rolled back Le Goues et al. (2012); Nguyen et al. (2013). Review can also extend over time or across tasks, such as by scheduling a monitoring task to inspect logs after a modified component has been in use for some time, or by identifying user behaviors from repeated requests across task trajectories (Du et al., 2017; Xu et al., 2010; Pan et al., 2026b).

Review uses methods such as reflection, active probing of the environment or test generation to verify a task outcome, or assessment of a harness revision against execution evidence accumulated over subsequent tasks (Madaan et al., 2023; Gou et al., 2024; Godefroid et al., 2005). When the available evidence is insufficient, review can be grounded in additional user input, such as by summarizing the unresolved issue and proactively asking a clarification question (Zhang et al., 2024c; Wang et al., 2025c; Tellex et al., 2014).

The lifecycle perception can capture the findings produced by review. These findings can ground further interaction with users, motivate lifecycle tasks, update the agent’s understanding of the user, or lead to revisions of its own components. Review and the lifecycle perception thus complete the feedback loop from

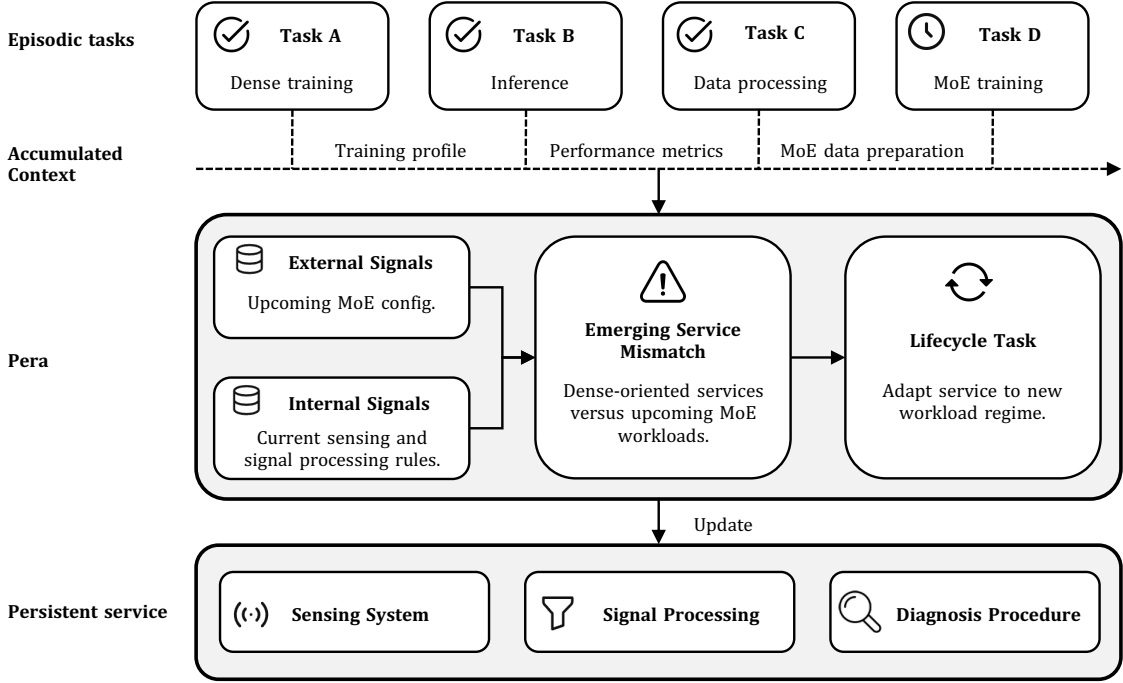


Figure 5: Illustration for a long-lived model-development setting. Interdependent episodic tasks accumulate context and introduce new workload requirements. Pera detects a mismatch between the evolving setting and the current reliability service, dispatches a lifecycle task to revise shared sensing and diagnostic procedures, and preserves the update for future episodic tasks.

task-level execution back into lifecycle-level decision-making, allowing evidence from individual tasks and across tasks to shape future service.

5 Case Study: Persistent Reliability for an Evolving Model-Development Workload

We use a model-development project on a shared computing cluster to illustrate how context, perception, and control shape continuing agent service. The same user performs a sequence of related but independently bounded workloads, while project artifacts, the computing environment, and the agent’s service procedures persist across them. This setting separates the completion of individual workloads from the maintenance of the shared conditions that support them over time.

Figure 5 summarizes the case. The upper sequence shows four episodic tasks whose artifacts, measurements, and operating experience accumulate as persistent context. Pera relates this context to external and internal signals and evaluates whether the existing reliability service remains appropriate as the workload changes. The transition toward Mixture-of-Experts (MoE) training exposes a mismatch between the upcoming workload and the existing sensing and diagnostic procedures, which triggers a lifecycle task that updates these shared procedures for subsequent workloads.

5.1 A Long-Lived Model-Development Setting

Consider a user developing a model through four successive stages on the same computing cluster. Dense training first produces a base model and its checkpoints. Inference and evaluation then produce performance measurements and error cases. These results guide a data-processing task that filters, transforms, and reorganizes training data. The processed data and earlier model artifacts are finally used in MoE training.

Each stage is an episodic task with its own objective and completion condition, but the stages belong to the same long-lived setting because their artifacts and operating conditions persist across task boundaries.

Table 1: Representative agent architectures organized by perception, control, and context. The comparison considers how each architecture supports a sequence of related episodic tasks as the computational requirements of the long-lived setting evolve.

System	Perception		Control		Context
	Scope	Source	Tasking	Decision-making	
Plan-and-Act Erdogan et al. (2025)	observation	external	episodic	task-level	transient
A-MEM Xu et al. (2026b)	observation	external	episodic	task-level	episodic
PaLM-E Driess et al. (2023)	observation	external	episodic	task-level	transient
ContextAgent Yang et al. (2025c)	observation/signal	external	proactive episodic	task-level	transient/episodic/persistent
Pera	observation/signal	external/internal	episodic/lifecycle	task-level/lifecycle-level	transient/episodic/persistent

Dense training produces the model consumed by inference; inference produces evidence that guides data processing; and data processing produces the dataset consumed by later training. Resource configurations, performance profiles, diagnostic results, user preferences, and reusable operating procedures also persist across the sequence.

By the end of data processing, the setting already shows signs of the next change. Accumulated project context records the characteristics of dense training, inference, and data processing, while the next model configuration and resource plan indicate a transition toward MoE workloads. At the same time, the agent’s monitoring baselines and diagnostic procedures still encode assumptions from the earlier workload regime. Figure 5 represents this condition as the emerging service mismatch: the workload is changing faster than the reliability service that interprets it. No current episodic task has failed; the relevant condition is that the persistent reliability service is no longer aligned with the workloads it must support.

We use this mismatch to compare the architectures below. The comparison asks whether each architecture preserves information across episodic tasks, perceives changes whose significance extends beyond the current objective, and turns those changes into work that maintains the service procedures shared by future tasks.

5.2 Architectural Comparison

Table 1 maps four representative systems and Pera onto the perception, control, and context dimensions introduced in Sections 3 and 4. Each architecture is considered under the same evolving project. This comparison isolates what each architecture preserves across the sequence, what changes it perceives, and what kind of work those changes produce.

Task-centered agent: Plan-and-Act. Plan-and-Act (Erdogan et al., 2025) organizes external observations and transient context around the current episodic objective. It plans and executes each stage when that stage is presented as a task: configure dense training, inspect an inference run, carry out data processing, or execute an MoE workload.

Its control ends at the boundary of the active task. Information retained during one execution serves that execution, and the next workload begins under a new task-level objective. Therefore, project-level dependencies and changes in reliability requirements remain outside the architecture’s persistent control and must be rediscovered within later episodic tasks.

Memory-augmented agent: A-MEM. A-MEM (Xu et al., 2026b) adds episodic context, allowing experience from one stage to remain available in later stages. Dense-training behavior, inference results, diagnostic observations, and prior user feedback remain retrievable when the project moves to data processing or later training. This creates continuity across the episodic sequence.

However, the retained experience remains an input to task-level decision-making. The emerging MoE workload becomes operationally relevant when a later task retrieves and uses that context. Memory preserves how the project reached its current state, but does not turn the changing workload regime into work that maintains the reliability service itself.

Sensor-grounded agent: PaLM-E. PaLM-E (Driess et al., 2023) extends the perception available within an episodic task through external sensor observations. In the model-development setting, this grounding provides information about accelerator, CPU, storage, and other system conditions relevant to the workload currently being executed.

These observations remain organized around the active task. They improve the agent’s understanding of the current workload, but do not preserve the accumulated project history or track how reliability requirements change across workload stages. Therefore, PaLM-E extends what the current task observes, whereas A-MEM extends what later tasks remember.

Context-aware proactive agent: ContextAgent. ContextAgent (Yang et al., 2025c) combines persistent context with continuing external perception. Therefore, the transition toward MoE workloads becomes visible before the user starts the next training task. The agent relates the upcoming configuration and resource plan to accumulated project context and proactively initiates an episodic service, such as assessing the expected workload or informing the user that operating conditions have changed.

Its control scope remains episodic. The perceived change produces another service task directed at the emerging workload, while the sensing and diagnostic machinery shared across workload stages remains outside the maintained service state. Proactivity changes when assistance begins, but does not extend control from individual service tasks to the persistent procedures that support them.

Persistent agent: Pera. Pera extends the scope of control from individual workloads to the reliability service shared across them. In Figure 5, external signals describe the transition toward MoE training, including the upcoming model configuration and resource plan, while internal signals expose the assumptions encoded in the agent’s existing sensing and diagnostic procedures. Combined with the accumulated context from earlier workloads, these signals reveal an emerging service mismatch: the project is moving toward a new workload regime, but the reliability service remains calibrated to the previous one.

This mismatch is significant because it concerns future service rather than the success of the current episodic task. Therefore, Pera turns it into a lifecycle task that revises the shared sensing, signal-processing, and diagnostic procedures. As illustrated in Figure 5, the lifecycle task does not replace or extend the upcoming MoE training task. Instead, it updates the persistent service used across workloads so that normal and abnormal behavior can be interpreted according to the new workload regime while preserving support for earlier stages.

Once validated, the revised procedures become part of the agent’s continuing service state. The subsequent MoE training remains an ordinary episodic task, but it is now served by a reliability service that has already adapted to the evolving project. Therefore, Pera connects two capabilities that the preceding architectures separate: signals make cross-task changes observable, and lifecycle tasks turn those changes into persistent adaptations that benefit future episodic tasks.

6 Actionable Insights

Beyond providing an architecture for characterizing persistent agents, Pera offers a lens for examining what existing agent systems already realize and what remains underdeveloped for building persistent agents that provide continuing service in long-lived settings. Organizing recent work through this lens reveals that many capabilities relevant to persistent agency have begun to emerge, but important gaps remain. Together with the preceding case study, these gaps suggest several actionable directions for advancing persistent agents.

Adaptive sensing: deciding what, when, and how to sense. A persistent agent can be exposed to a continuous stream of signals from people, physical environments, digital systems, and the agent itself. Recent work, including ContextAgent (Yang et al., 2025c) and ProAgent (Yang et al., 2025b), addresses parts of this problem through sensory context and on-demand perception (Pu et al., 2025; Lee et al., 2025). Yet sensing over a long-lived setting raises a broader question: how can an agent continually decide what is worth perceiving as its setting and information needs change? Adaptive sensing keeps the agent aware of relevant changes without processing every available signal continuously.

-
- **Broad sensing scope.** Useful signals can take very different forms, such as a change in temperature, newly created files, or a change in a user’s tone of voice. Therefore, a general lifecycle-perception interface should support signals from human, physical, digital, and internal sources rather than assume a single language or software interface (Yang et al., 2025a; Zhang et al., 2026g). Broadening the sensing interface can expose more signals relevant to future service and extend language agents beyond purely digital environments into the physical world.
 - **Selective sensing.** Signals in a long-lived setting can be abundant, noisy, and unevenly useful, making continuous processing of everything both unnecessary and computationally expensive. A promising direction is to adapt which sources are monitored, when they are inspected, and at what frequency and granularity. For instance, routine periods require only lightweight monitoring, while an emerging anomaly can trigger finer-grained or more expensive sensing. Such selective allocation can make continuing perception more scalable while reducing unnecessary computation.
 - **Evolving sensors.** Beyond selecting among available sensing sources, a persistent agent should also evolve how those sources are sensed as the setting changes (Li et al., 2026b). As the environment and relevant signals change over time, the agent should revise its software sensors, add new detectors or data sources, and use grounding actions to recalibrate, repair, replace, or introduce physical sensors. This makes the agent’s sensing capability itself evolvable over its lifetime.

Signal representation: learning how to describe change. Sensing exposes events, but lifecycle-level reasoning needs a representation in which perceived changes can be compared and related over time. Natural language is flexible, yet similar conditions can be described in very different ways, making long-term comparison and aggregation difficult (Liu et al., 2023; Jia et al., 2025). A promising direction is to map sensed events into more structured representations that need not themselves be natural language and can evolve with the setting.

- **Domain-specific signal languages.** Consider an agent managing a training cluster. During one run, several workers slow down, communication latency rises, and job throughput drops. Although these events together indicate the same underlying condition, such as a communication bottleneck, natural-language descriptions can vary substantially across runs. Therefore, recognizing a similar condition weeks later requires more than matching surface descriptions. A promising approach is to map sensed events into a domain-specific signal language with shared concepts and relations. Such representations can provide a more consistent basis for comparing signals and reasoning over them across sources and over time.
- **Cross-signal aggregation.** Some changes become visible only when multiple signals are considered together. A single execution failure does not establish a persistent problem, whereas repeated failures across tasks provide evidence that a previously effective procedure has become obsolete. Therefore, a promising direction is to develop mechanisms that relate evidence across events and determine when multiple signals together indicate a higher-level condition (Etzion & Niblett, 2010). This is especially important for weak signals that are individually insufficient but become informative when they recur over time. Aggregating such evidence over long periods enables persistent agents to interpret persistent changes that cannot be understood from individual signals alone.

Context learning: thinking beyond reasoning over static model knowledge and curated context. Context learning is a key capability for persistent agents. Over a long-lived setting, an agent accumulates large amounts of context that can be diverse, messy, and unfamiliar to the model, rather than a clean context prepared for a single task (Dou et al., 2026b). Therefore, the challenge goes beyond reasoning over static model knowledge, as persistent agents need to continually learn from the context and use what they learn across future tasks.

- **Learning from real-world context.** Real-world context rarely comes in a single, well-organized form. For example, an employee’s work context is scattered across conversations, documents, previous

tasks, and application states, with useful information distributed across them rather than collected in one place (Chen et al., 2026b; Wei et al., 2026). A promising direction is to develop agents in environments where context is not cleanly packaged into a single document or conversation but instead accumulates across diverse sources and interactions over time. Doing so can bring agents closer to understanding the same messy information environment in which people work and make decisions.

- **Learning new knowledge from context.** Much of the context encountered over an agent’s lifetime contains knowledge that is new to the model, such as an unfamiliar workflow, a newly introduced policy, or a procedure learned through experience (Si et al., 2026). CL-bench (Dou et al., 2026c;b) begins to isolate this capability by requiring models to learn and apply knowledge, rules, and procedures absent from pre-training, while showing that substantial room for improvement remains. Developing models that can reliably learn such new knowledge is a promising direction, enabling them to reason over unfamiliar knowledge from context rather than relying only on what is already learned in parameters.
- **Learning the temporal validity of context.** Knowledge learned from context does not remain valid indefinitely (Huang et al., 2026b). Its validity can change over time, for example, when an employee moves to a new project, a company policy is revised, or an earlier understanding is corrected by new evidence (Sun et al., 2026a). A promising direction is to make temporal validity part of context learning, so that agents learn not only what a piece of context means, but also when it applies and how later evidence changes its validity (Su et al., 2026). Recognizing such changes can further expose lifecycle-relevant signals that affect future service.

Proactivity: thinking beyond initiative within a given task. Recent agent research has increasingly focused on long-horizon task execution, where an agent is given a task and context and must solve a complex objective. However, for persistent agents, an equally important challenge is determining when perceived changes and accumulated context warrant initiative, even without an explicitly specified task. Proactivity can extend across different stages of persistent service, such as uncovering emerging needs from accumulated context, preparing or initiating useful work, and improving the agent itself when its current capabilities no longer fit future service. Existing work has explored parts of this spectrum, such as proactively clarifying or acquiring information for an ongoing task (Zhang et al., 2024c) and, more recently, anticipating future needs from persistent context (Lu et al., 2025; Pasternak et al., 2025; Xie et al., 2026).

- **Discovering and anticipating needs from accumulated context.** A promising direction is to enable persistent agents to proactively infer emerging needs from accumulated context (Tang et al., 2026b; Guo et al., 2026b; Sun et al., 2026b) and newly perceived signals, rather than waiting for explicit requests. For example, an agent assisting an employee over months learns about their ongoing projects, deadlines, recurring information needs, and typical ways of working. A new document or a change in the project may reveal an opportunity for useful follow-up before the employee explicitly requests assistance (Pasternak et al., 2025; Yang et al., 2025c). Recent work (Tang et al., 2026b; Sun et al., 2026b) has begun to explore such context-driven discovery, but it remains underexplored in long-lived settings, where some needs become apparent only across repeated interactions and tasks. Combined with increasingly capable long-horizon execution, further progress could move agents from solving given tasks toward autonomously identifying and carrying out what needs to be done.
- **Deciding when and how to act on anticipated needs.** Identifying a possible need does not determine whether the agent should act. For persistent agents that repeatedly take initiative, acting too aggressively can create interruptions or risks, while being overly conservative can make anticipation ineffective. A promising direction is to calibrate proactive action based on factors such as confidence, expected benefit, and risk, with responses ranging from continued observation or background preparation to asking the user or carrying out low-risk actions Horvitz (1999). User feedback on previous initiatives can further help the agent learn which proactive behaviors are useful for a particular user and setting. When an anticipated need instead reveals a capability gap, proactive preparation can include maintaining or improving the agent before the limitation affects future service.

Lifecycle control: coordinating work beyond a single task loop. Most agent control focuses on deciding what to do next within a given task. Persistent agents additionally need to coordinate work across tasks, as episodic and lifecycle tasks can differ in urgency, depend on one another, compete for resources, and involve modifications to persistent components reused by future tasks (Wei et al., 2025). This motivates lifecycle-level control over how work is coordinated and governed over time, including how tasks are prioritized and how persistent changes are specified and accepted.

- **Lifecycle-aware dispatch.** Dispatching work in a long-lived setting differs from ordinary resource scheduling because tasks can have different implications. Episodic and lifecycle tasks differ in urgency and long-term impact: user-facing work often carries immediate response requirements, whereas lifecycle work can determine the quality of many future tasks. A promising direction is to make dispatch sensitive to the service implications of each task, such as its urgency, dependencies, and long-term impact. Better lifecycle-aware dispatch could balance immediate responsiveness with long-term service quality, ensuring that important maintenance and adaptation are not indefinitely postponed without unnecessarily delaying urgent user-facing work.
- **Specifying acceptable lifecycle changes.** Because lifecycle tasks can modify components reused by future executions, an objective alone is insufficient to specify an acceptable change. A promising direction is for the agent itself to specify, as part of the lifecycle task package, not only what should be achieved but also the conditions under which the resulting change can be accepted, such as its allowed scope, required evidence, and recovery requirements (Chen et al., 2026d). This gives lifecycle control and review a clearer basis for deciding whether an update should be accepted or further revised (Santos-Grueiro, 2026). Reusable specifications for common forms of lifecycle change could make persistent adaptation more reliable across tasks (Zhang & Zhang, 2026).

Agent self-improvement: model and harness. Agent self-improvement concerns how accumulated context is translated into persistent changes to the agent itself. Recent work increasingly uses *agent harness* to refer to the non-model parts within an agent (Huang et al., 2026c; Pan et al., 2026a). Recent work has begun to enable agents to improve parts of their own harnesses, such as skills, tools, and memory mechanisms (Chen et al., 2026a; Lin et al., 2026; Cai et al., 2026; Lee et al., 2026). In Pera, the persistent agent core helps the agent proactively identify when change is needed. Such improvement can target both the harness and the model. The former provides an explicit and flexible space for rapid adaptation, while the latter can internalize repeatedly useful improvements into its parameters. Despite their different forms, both can be improved along similar dimensions, including long-lived knowledge, decision-making procedures, and capabilities such as reasoning and learning.

- **Improving the improvement process.** Self-improvement itself can become a capability that improves through accumulated context (Chen et al., 2026a; Lin et al., 2026). Evidence accumulated across tasks and over time can be used not only to change how the agent operates, but also to refine how it identifies and responds to the need for change. The adaptive sensing discussed earlier provides one example: improving what, when, and how the agent senses alters the evidence available to lifecycle control. Accumulated experience can likewise refine how lifecycle control responds, such as when to take initiative, which changes warrant intervention, and how much autonomy to exercise. Making these mechanisms themselves evolvable could help the agent become better at identifying when and how to improve itself. Compared with the evolution of individual harness components, this meta-level remains underexplored for persistent agents.
- **Coordinated evolution.** Existing self-improvement methods often target particular parts of the agent harness, but improvements to different components can interact in nontrivial ways (Zhang et al., 2026a; Cai et al., 2026; Lee et al., 2026). For example, evolving a sensor exposes new signals that existing lifecycle control cannot yet use effectively, while introducing a new tool requires corresponding changes to the decision procedure that invokes it. Therefore, a promising direction is to move from isolated component evolution toward coordinated evolution across the agent. Such coordination can help improvements discovered in one component translate into better overall behavior rather than creating new mismatches elsewhere in the system.

- **Safe self-improvement.** Recent work has begun to validate self-generated agent modifications using execution feedback or regression testing (Chen et al., 2026a; Zhang et al., 2026a). For persistent agents, the stakes are higher because an accepted modification can persist and affect many future tasks. So calibrating the degree of autonomous modification according to the risk and reversibility of the change is a promising research direction. Reversible changes can be explored more freely, for example through version-controlled updates (e.g., Git) that can be tested and rolled back, whereas more consequential modifications can require broader regression testing or staged acceptance, enabling safer and more autonomous self-improvement over time.
- **Model self-improvement.** The harness provides a flexible space for agent self-improvement, where new knowledge and decision procedures can be introduced and refined directly. A further opportunity is to use such accumulated improvements to evolve the model itself. Early work on agent fine-tuning has begun to transfer behaviors learned from agent trajectories into model parameters (Lu et al., 2026; Zeng et al., 2024), but the broader connection between harness evolution and model learning remains underexplored. A promising direction is for agents to self-improve their harnesses to discover and validate better decision procedures and learning and reasoning strategies, and then internalize repeatedly useful improvements into the model. In this way, persistent experience could improve not only what the model knows, but also how it reasons and learns.

Agent evaluation: thinking beyond long-horizon tasks. Most agent evaluations still center on individual tasks, increasingly including tasks with longer and more complex execution (i.e., long-horizon tasks) (Starace et al., 2025; Kwa et al., 2026; Xu et al., 2026a). Some work (Zheng et al., 2025a; Dou et al., 2025; Wang et al., 2026a) begins to move beyond isolated tasks by evaluating skill acquisition and transfer across interdependent task sequences. Looking ahead, we expect agent evaluation to develop along several dimensions: toward tasks with greater real-world value, toward long-horizon execution within a task, and toward continuing service across the lifetime of a long-lived setting.

However, existing benchmarks still do not capture the full trajectory of long-lived service, where goals evolve over time, context continually accumulates and changes, and the agent must proactively support diverse tasks and adapt itself over time. For example, in a knowledge-intensive professional setting, an agent assistant should support staff in their work over extended periods by working with accumulated and evolving context, supporting changing projects and needs, proactively identifying useful work, and adapting itself as the setting changes. Such evaluation shifts the focus from *how long an agent can execute a task* to how well it can sustain and improve service throughout extended real-world use. Realistic and scalable benchmarks for this form of persistent service remain underdeveloped.

7 Discussion

In this section, we discuss two lines of work closely related to our conceptual framework.

Continual learning and lifelong learning. We first consider two learning paradigms concerned with the continuous adaptation of models and agents. Continual learning (Parisi et al., 2019; De Lange et al., 2021; Shi et al., 2025) studies how models acquire knowledge from evolving data or task streams while retaining previously learned capabilities (Zheng et al., 2025b). Much of the work focuses on updating model parameters while mitigating catastrophic forgetting (McCloskey & Cohen, 1989; Kirkpatrick et al., 2017; Rebuffi et al., 2017). We distinguish it from lifelong learning, which adopts the broader goal of accumulating reusable knowledge and experience across a lifetime (Zheng et al., 2025a; Cai et al., 2025). In the context of language models, prior work has studied how evolving knowledge and past experience can be incorporated into model parameters or external knowledge stores (Lewis et al., 2020; Borgeaud et al., 2022; Meng et al., 2022a;b). Recent work has extended this perspective to language agents, examining how their memory, skills, and actions continually adapt through interaction with changing environments (Zhao et al., 2024; Cao et al., 2026; Zhang et al., 2026b; Yang et al., 2026b).

However, these works leave several gaps. First, existing formulations (Yang et al., 2026d;c) center on streams of tasks, where experience from completed tasks is retained and reused to improve performance on subsequent

tasks. Yet realizing persistent agents requires a broader perceptual scope, in which agents continually and proactively perceive service-relevant signals from both external and internal sources, including changes that do not arrive as explicit tasks (Deng et al., 2024; Lu et al., 2025; Yang et al., 2025c). Moreover, persistent agents must also operate prospectively, continually uncovering new contextual information as the setting evolves and using it to adapt their ongoing assistance. Finally, existing work on persistent agents follows two main lines: either proposing specific lifelong learning approaches centered on learning from past experience, or surveying the broader research landscape by organizing existing work into taxonomies of agent capabilities and components (Gao et al., 2025; Cai et al., 2025). Neither line, however, provides a conceptual architecture that both characterizes the operating principles of persistent agents and offers a unified lens for understanding how existing efforts contribute to them. Pera addresses these gaps by proposing a perception-centered conceptual architecture and using it to retrospectively organize recent work.

Frameworks for agents. A long line of work (Maes, 1994; Wooldridge & Jennings, 1995; Masterman et al., 2024) has proposed general architectures and conceptual frameworks for organizing intelligent agents. Early work in symbolic AI developed cognitive architectures such as Soar (Laird et al., 1987), which integrates perception, memory, learning, decision-making, and action within a unified cognitive system. The Common Model of Cognition (Laird et al., 2017) later summarized the components and processing cycles shared by Soar and other influential cognitive architectures. Intentional architectures such as BDI (Rao et al., 1995; De Silva et al., 2020) instead organize practical reasoning around beliefs, desires, intentions, and plans.

With the rise of large language models, architectural thinking has been extended to language agents in multiple directions (Wang et al., 2024a; Xi et al., 2025). CoALA (Sumers et al., 2023) brings the cognitive-architecture perspective to language agents by structuring them around memory, internal and external actions, and decision procedures, and uses this structure to categorize existing agent methods. Recent work has also explored the system infrastructure for language agents. AIOS (Ge et al., 2023; Mei et al., 2024) organizes its runtime through an operating-system-like architecture that separates agent applications from the kernel responsible for execution and resource management. Collectively, these language-agent frameworks organize particular aspects of agents, but do not provide a lifecycle-level account of how an agent continually perceives service-relevant changes and sustains evolving service responsibilities in a long-lived setting.

Another line of work places perception and adaptive control at the center of agent architectures. The subsumption architecture (Brooks, 1986; 1991) tightly couples sensory inputs with layered behaviors, enabling embodied agents to respond continuously to changes in their environment. Hayes-Roth’s architecture for adaptive intelligent systems (AIS) (Hayes-Roth, 1995) further supports the dynamic adaptation of perceptual strategies, reasoning tasks and methods, and control plans as operating conditions change. The AIS architecture directly inspired Pera’s emphasis on perception and control.

However, in these architectures, perception primarily serves to regulate the agent’s behavior in support of current tasks. For persistent agents, perception serves a broader purpose, including identifying emerging needs and determining how the agent’s service should evolve with the setting (Zheng et al., 2026a). This difference in purpose leads to distinct designs for perception and control, and ultimately to different agent architectures, one oriented toward adapting current operation and the other toward sustaining and evolving service throughout the lifetime of a setting. Moreover, these works predate the emergence of language models and do not explain how their underlying design principles should be reinterpreted and extended for language agents. Pera builds on this tradition by making persistent and proactive assistance in long-lived settings a central architectural concern. It characterizes how external and internal signals give rise to lifecycle tasks, how lifecycle and episodic tasks interact, and how their interaction sustains and adapts the agent’s service over time.

8 Conclusion

In this paper, we introduced Pera, a perception-centered architecture for persistent agents. Pera extends existing cognitive frameworks for language agents by centering continual perception and lifecycle control, enabling adaptation across changing, long-lived settings. We also used Pera to systematically organize a broad body of recent work and to present actionable insights for future research. We hope Pera can provide a

conceptual and architectural foundation for agents in the large, supporting the development of proactive, long-lived, and adaptive intelligent systems.

References

- Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Wang. Agent s: An open agentic framework that uses computers like a human. In *International Conference on Learning Representations*, volume 2025, pp. 22924–22946, 2025.
- Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- Ruzena Bajcsy. Active perception. *Proceedings of the IEEE*, 76(8):966–1005, 1988.
- Ruzena Bajcsy, Yiannis Aloimonos, and John K Tsotsos. Revisiting active perception. *Autonomous Robots*, 42(2):177–196, 2018.
- Haonan Bian, Zhiyuan Yao, Sen Hu, Zishan Xu, Shaolei Zhang, Yifu Guo, Ziliang Yang, Xueran Han, Huacan Wang, and Ronghao Chen. Realmem: Benchmarking llms in real-world memory-driven interaction. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 14349–14365, 2026.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pp. 2206–2240. PMLR, 2022.
- Rodney Brooks. A robust layered control system for a mobile robot. *IEEE journal on robotics and automation*, 2(1):14–23, 1986.
- Rodney A Brooks. Intelligence without representation. *Artificial intelligence*, 47(1-3):139–159, 1991.
- Qianshu Cai, Yonggang Zhang, Xianzhang Jia, Huajiang Zheng, Wei Xue, Jun Song, Xinmei Tian, and Yike Guo. Moss: Self-evolution through source-level rewriting in autonomous agent systems. *arXiv preprint arXiv:2605.22794*, 2026.
- Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. In *International Conference on Learning Representations*, volume 2024, pp. 54067–54089, 2024.
- Yuxuan Cai, Yipeng Hao, Jie Zhou, Hang Yan, Zhikai Lei, Rui Zhen, Zhenhua Han, Yutao Yang, Junsong Li, Qianjun Pan, et al. Building self-evolving agents via experience-driven lifelong learning: A framework and benchmark. *arXiv preprint arXiv:2508.19005*, 2025.
- Zouying Cao, Jiaji Deng, Li Yu, Weikang Zhou, Zhaoyang Liu, Bolin Ding, and Hai Zhao. Remember me, refine me: A dynamic procedural memory framework for experience-driven agent evolution. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 16803–16822, 2026.
- Joyce Y Chai, Qiaozi Gao, Lanbo She, Shaohua Yang, Sari Saba-Sadiya, and Guangyue Xu. Language to action: Towards interactive task learning with physical agents. In *IJCAI*, volume 7, pp. 2–9, 2018.
- Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. Chateval: Towards better llm-based evaluators through multi-agent debate. In *International conference on learning representations*, volume 2024, pp. 9079–9093, 2024.
- Hanxiang Chao, Yihan Bai, Rui Sheng, Tianle Li, and Yushi Sun. Stale: Can llm agents know when their memories are no longer valid? *arXiv preprint arXiv:2605.06527*, 2026.
- Guoxin Chen, Zhong Zhang, Xin Cong, Fangda Guo, Yesai Wu, Yankai Lin, Wenzheng Feng, and Yasheng Wang. Learning evolving tools for large language models. In *International Conference on Learning Representations*, volume 2025, pp. 87140–87169, 2025.

-
- Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable llm agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*, 2026a.
- Yifan Chen, Fei Yin, Qingyan Bai, Zicheng Lin, and Yujiu Yang. Mmcl-bench: Multimodal context learning from visual rules, procedures, and evidence. *arXiv preprint arXiv:2605.12703*, 2026b.
- Yuxin Chen, Yi Zhang, Zhengzhou Cai, Yaorui Shi, Zhiyuan Yao, Chenhang Cui, Jingnan Zheng, Yaqi Huo, Xi Su, Qi Gu, et al. Vitabench 2.0: Evaluating personalized and proactive agents in long-term user interactions. *arXiv preprint arXiv:2605.27141*, 2026c.
- Zheng Chen, Hanqing Liu, Duling Xu, Dong Dong, Jialin Li, Bangzheng Pu, and Jidong Zhai. Cordon: Semantic transactions for tool-using llm agents. *arXiv preprint arXiv:2606.17573*, 2026d.
- Zhiling Chen, David Gorsich, Matthew P Castanier, Yang Zhang, Jiong Tang, and Farhad Imani. Task-aware scanning parameter configuration for robotic inspection using vision language embeddings and hyperdimensional computing. *arXiv preprint arXiv:2605.03909*, 2026e.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Sadia Sultana Chowa, Riasad Alvi, Subhey Sadi Rahman, Md Abdur Rahman, Mohaimenul Azam Khan Raiaan, Md Rafiqul Islam, Mukhtar Hussain, and Sami Azam. From language to action: a review of large language models as autonomous agents and tool users. *Artificial Intelligence Review*, 59(2):71, 2026.
- Michael T Cox. Metacognition in computation: A selected research review. *Artificial intelligence*, 169(2): 104–141, 2005.
- Gianpaolo Cugola and Alessandro Margara. Processing flows of information: From data stream to complex event processing. *ACM Computing Surveys (CSUR)*, 44(3):1–62, 2012.
- Gautier Dagan, Frank Keller, and Alex Lascarides. Dynamic planning with a llm. *arXiv preprint arXiv:2308.06391*, 2023.
- Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE transactions on pattern analysis and machine intelligence*, 44(7):3366–3385, 2021.
- Rogério De Lemos, Holger Giese, Hausi A Müller, Mary Shaw, Jesper Andersson, Marin Litoiu, Bradley Schmerl, Gabriel Tamura, Norha M Villegas, Thomas Vogel, et al. Software engineering for self-adaptive systems: A second research roadmap. In *Software Engineering for Self-Adaptive Systems II: International Seminar, Dagstuhl Castle, Germany, October 24-29, 2010 Revised Selected and Invited Papers*, pp. 1–32. Springer, 2013.
- Lavindra De Silva, Felipe Rech Meneguzzi, and Brian Logan. Bdi agent architectures: A survey. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI), 2020, Japão.*, 2020.
- Yang Deng, Lizi Liao, Zhonghua Zheng, Grace Hui Yang, and Tat-Seng Chua. Towards human-centered proactive conversational agents. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*, pp. 807–818, 2024.
- Frank DeRemer and Hans H Kron. Programming-in-the-large versus programming-in-the-small. *IEEE Transactions on Software Engineering*, (2):80–86, 1976.
- Deming Ding, Shichun Liu, Enhui Yang, Jiahang Lin, Ziyang Chen, Shihan Dou, Honglin Guo, Weiyu Cheng, Pengyu Zhao, Chengjun Xiao, et al. Octobench: Benchmarking scaffold-aware instruction following in repository-grounded agentic coding. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5958–5978, 2026.

-
- Shihan Dou, Yan Liu, Haoxiang Jia, Enyu Zhou, Limao Xiong, Junjie Shan, Caishuang Huang, Xiao Wang, Xiaoran Fan, Zhiheng Xi, et al. StepCoder: improving code generation with reinforcement learning from compiler feedback. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 4571–4585, 2024.
- Shihan Dou, Ming Zhang, Chenhao Huang, Jiayi Chen, Feng Chen, Shichun Liu, Yan Liu, Chenxiao Liu, CHENG ZHONG, Zongzhang Zhang, Tao Gui, Chao Xin, Wei Chengzhi, Lin Yan, Qi Zhang, and Xuanjing Huang. EvalLearn: Quantifying the learning capability and efficiency of LLMs via sequential problem solving. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Muling Wu, Yunbo Tao, Ming Zhang, Mingxu Chai, Jessica Fan, Zhiheng Xi, et al. What is wrong with your code generated by large language models? an extensive study. *Science China Information Sciences*, 69(1):112107, 2026a.
- Shihan Dou, Yujiong Shen, Chenhao Huang, Junjie Ye, Jiayi Chen, Junzhe Wang, Qianyu He, Shichun Liu, Changze Lv, Jiahang Lin, et al. Cl-bench life: Can language models learn from real-life context? *arXiv preprint arXiv:2604.27043*, 2026b.
- Shihan Dou, Ming Zhang, Zhangyue Yin, Chenhao Huang, Yujiong Shen, Junzhe Wang, Jiayi Chen, Yuchen Ni, Junjie Ye, Cheng Zhang, et al. Cl-bench: A benchmark for context learning. *arXiv preprint arXiv:2602.03587*, 2026c.
- Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.
- Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS '17*, pp. 1285–1298, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450349468. doi: 10.1145/3133956.3134015.
- Shangheng Du, Jiabao Zhao, Jinxin Shi, Zhentao Xie, Xin Jiang, Yanhong Bai, and Liang He. A survey on the optimization of large language model-based agents. *ACM Computing Surveys*, 58(9):1–37, 2026.
- Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Plan-and-act: Improving planning of agents for long-horizon tasks. *arXiv preprint arXiv:2503.09572*, 2025.
- O. Etzion and P. Niblett. *Event Processing in Action*. Manning, 2010. ISBN 9781935182214.
- Haodi Fan and Zucong Lan. From cognitive architectures to language agents: A mechanism-level review of lineage, convergence, and migration gaps. *arXiv preprint arXiv:2607.23942*, 2026.
- Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, et al. A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems. *arXiv preprint arXiv:2508.07407*, 2025.
- Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjuan Zhong. Retool: Reinforcement learning for strategic tool use in llms. In *International Conference on Learning Representations*, volume 2026, pp. 37909–37926, 2026.
- Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, et al. Agentscope: A flexible yet robust multi-agent platform. *arXiv preprint arXiv:2402.14034*, 2024a.
- Ge Gao, Alexey Taymanov, Eduardo Salinas, Paul Mineiro, and Dipendra Misra. Aligning llm agents by learning latent preference from user edits. *Advances in neural information processing systems*, 37: 136873–136896, 2024b.

-
- Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, et al. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *arXiv preprint arXiv:2507.21046*, 2025.
- Minghe Gao, Juncheng Li, Yuze Lin, Xuqi Liu, Jiaming Ji, Xiaoran Pan, Zihan Xu, Xian Li, Mingjie Li, Wei Ji, et al. Arcadia: Toward a full-lifecycle framework for embodied lifelong learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1031–1040, 2026.
- David Garlan, Mary Shaw, et al. An introduction to software architecture. *Advances in software engineering and knowledge engineering*, 1(3.4), 1993.
- David Garlan, Robert Allen, and John Ockerbloom. Architectural mismatch or why it’s hard to build systems out of existing parts. In *Proceedings of the 17th international conference on Software engineering*, pp. 179–185, 1995.
- David Garlan, S-W Cheng, A-C Huang, Bradley Schmerl, and Peter Steenkiste. Rainbow: Architecture-based self-adaptation with reusable infrastructure. *Computer*, 37(10):46–54, 2004.
- Yingqiang Ge, Yujie Ren, Wenyue Hua, Shuyuan Xu, Juntao Tan, and Yongfeng Zhang. Llm as os, agents as apps: Envisioning aios, agents and the aios-agent ecosystem. *arXiv preprint arXiv:2312.03815*, 2023.
- Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Quentin Carbonneaux, Taco Cohen, and Gabriel Synnaeve. Rlef: Grounding code llms in execution feedback with reinforcement learning. *arXiv preprint arXiv:2410.02089*, 2024.
- Michael P. Georgeff and Francois Felix Ingrand. Decision-making in an embedded reasoning system. In *Proceedings of the 11th International Joint Conference on Artificial Intelligence - Volume 2, IJCAI’89*, pp. 972–978, San Francisco, CA, USA, 1989. Morgan Kaufmann Publishers Inc.
- Cristiano Giuffrida, Calin Iorgulescu, Anton Kuijsten, and Andrew S Tanenbaum. Back to the future: Fault-tolerant live update with time-traveling state transfer. In *27th Large Installation System Administration Conference (LISA 13)*, pp. 89–104, 2013.
- Patrice Godefroid, Nils Klarlund, and Koushik Sen. Dart: directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI ’05*, pp. 213–223, New York, NY, USA, 2005. Association for Computing Machinery. ISBN 1595930566. doi: 10.1145/1065010.1065036.
- Boyuan Gou, Demi Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for gui agents. In *International Conference on Learning Representations*, volume 2025, pp. 30851–30883, 2025.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Yang, Nan Duan, Weizhu Chen, et al. Critic: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, volume 2024, pp. 57734–57811, 2024.
- Qiao Gu, Ali Kuwajerwala, Sacha Morin, Krishna Murthy Jatavallabhula, Bipasha Sen, Aditya Agarwal, Corban Rivera, William Paul, Kirsty Ellis, Rama Chellappa, et al. Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5021–5028. IEEE, 2024.
- Dongxin Guo, Jikun Wu, and Siu Ming Yiu. Saga: Workflow-atomic scheduling for ai agent inference on gpu clusters. *arXiv preprint arXiv:2605.00528*, 2026a.
- Honglin Guo, Qi Zhang, Yu Zhang, Weijie Li, Rui Zheng, Zhikai Lei, Qiyuan Peng, Zhiheng Xi, Tao Gui, and Qi Zhang. Agora: An archive-grounded benchmark for agentic workplace document reasoning, 2026b. URL <https://arxiv.org/abs/2606.24526>.
- Shuchen Guo, Ehsan Latif, Yifan Zhou, Xuan Huang, and Xiaoming Zhai. Using generative ai and multi-agents to provide automatic feedback. *ArXiv*, abs/2411.07407, 2024.

-
- Daniel Gyllstrom, Eugene Wu, Hee-Jin Chae, Yanlei Diao, Patrick Stahlberg, and Gordon Anderson. Sase: Complex event processing over streams. *arXiv preprint cs/0612128*, 2006.
- Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu. Reasoning with language model is planning with world model. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pp. 8154–8173, 2023.
- Barbara Hayes-Roth. An architecture for adaptive intelligent systems. *Artificial intelligence*, 72(1-2):329–365, 1995.
- Barbara Hayes-Roth, Richard Washington, David Ash, Rattikorn Hewett, Anne Collinot, Angel Vina, and Adam Seiver. Guardian: A prototype intelligent agent for intensive-care monitoring. *Artificial Intelligence in Medicine*, 4(2):165–185, 1992.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Steven Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, volume 2024, pp. 23247–23275, 2024a.
- Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. Cogagent: A visual language model for gui agents. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14281–14290. IEEE, 2024b.
- Eric Horvitz. Principles of mixed-initiative user interfaces. In *Proceedings of the SIGCHI conference on Human Factors in Computing Systems*, pp. 159–166, 1999.
- Petr Hosek and Cristian Cadar. Safe software updates via multi-version execution. In *2013 35th International Conference on Software Engineering (ICSE)*, pp. 612–621. IEEE, 2013.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, volume 2025, pp. 21344–21377, 2025a.
- Yihao Hu, Zhihao Wen, Xiujiu Liu, Pan Wang, Xin Zhang, and Wei Wu. Seal: Synergistic co-evolution of agents and learning environments. *arXiv preprint arXiv:2605.24426*, 2026.
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, et al. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564*, 2025b.
- Donghao Huang, Joon Kiat Chua, and Zhaoxia Wang. Beyond task success: Measuring workflow fidelity in llm-based agentic payment systems. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pp. 357–362. Springer, 2026a.
- Wei-Chieh Huang, Weizhi Zhang, Yueqing Liang, Yuanchen Bei, Yankai Chen, Tao Feng, Zhen Tan, Yu Wang, Tianxin Wei, Shanglin Wu, et al. A survey of agent memory in the second half: Towards self-evolving and long-horizon agents. *Transactions on Machine Learning Research*, 2026b.
- Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, pp. 9118–9147. PMLR, 2022.
- Wenlong Huang, Fei Xia, Dhruv Shah, Danny Driess, Andy Zeng, Yao Lu, Pete Florence, Igor Mordatch, Sergey Levine, Karol Hausman, et al. Grounded decoding: Guiding text generation with grounded models for embodied agents. *Advances in Neural Information Processing Systems*, 36:59636–59661, 2023a.
- Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Tomas Jackson, Noah Brown, Linda Luu, Sergey Levine, Karol Hausman, and brian ichter. Inner monologue: Embodied reasoning through planning with language models. In Karen Liu, Dana Kulic, and Jeff Ichnowski (eds.), *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pp. 1769–1782. PMLR, 14–18 Dec 2023b.

-
- Yue Huang, Wenjie Wang, Han Bao, Yuchen Ma, Xiaonan Luo, Yi Nian, Haomin Zhuang, Zheyuan Liu, Yue Zhao, and Xiangliang Zhang. Memoharness: Agent harnesses that learn from experience. *arXiv preprint arXiv:2607.14159*, 2026c.
- Md Farhan Ishmam and Kenneth Marino. Timewarp: Evaluating web agents by revisiting the past. *arXiv preprint arXiv:2603.04949*, 2026.
- Haoxiang Jia, Robbie Morris, He Ye, Federica Sarro, and Sergey Mechtaev. Automated repair of ambiguous problem descriptions for llm-based code generation. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 367–379. IEEE, 2025.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pp. 6769–6781, 2020.
- Timo Kaufmann, Paul Weng, Viktor Bengs, and Eyke Hüllermeier. A survey of reinforcement learning from human feedback. *arXiv preprint arXiv:2312.14925*, 2023.
- J.O. Kephart and D.M. Chess. The vision of autonomic computing. *Computer*, 36(1):41–50, 2003. doi: 10.1109/MC.2003.1160055.
- Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, pp. 39–48, 2020.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.
- Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, et al. Measuring ai ability to complete long software tasks. *Advances in Neural Information Processing Systems*, 38:92213–92266, 2026.
- John E Laird, Allen Newell, and Paul S Rosenbloom. Soar: An architecture for general intelligence. *Artificial intelligence*, 33(1):1–64, 1987.
- John E Laird, Christian Lebiere, and Paul S Rosenbloom. A standard model of the mind: Toward a common computational framework across artificial intelligence, cognitive science, neuroscience, and robotics. *Ai Magazine*, 38(4):13–26, 2017.
- Tianwei Lan, Jiaqi Wu, Zeming Liu, Zhaoxin Fan, Haifeng Wang, and Yuhang Guo. Peap: Proactive embodied action sequence planning with joint understanding of vision and audio perception. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 23118–23138, 2026.
- Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. Genprog: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1):54–72, 2012. doi: 10.1109/TSE.2011.104.
- Geonsun Lee, Min Xia, Nels Numan, Xun Qian, David Li, Yanhe Chen, Achin Kulshrestha, Ishan Chatterjee, Yinda Zhang, Dinesh Manocha, et al. Sensible agent: A framework for unobtrusive interaction with proactive ar agents. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–22, 2025.
- Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. A human-inspired reading agent with gist memory of very long contexts. *arXiv preprint arXiv:2402.09727*, 2024.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.

-
- Meir M Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9): 1060–1076, 1980.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- Guanzhen Li, Liangming Pan, and Leye Wang. Proevent: An event-centric benchmark for proactive agents. *arXiv preprint arXiv:2607.17701*, 2026a.
- Hao Li, Chenghao Yang, An Zhang, Yang Deng, Xiang Wang, and Tat-Seng Chua. Hello again! llm-powered personalized agent for long-term dialogue. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 5259–5276, 2025a.
- Jialiang Li, Yi Qiao, Yunhan Guo, Changwen Chen, and Wenzhao Lian. Act, sense, act: Learning non-markovian active perception strategies from large-scale egocentric human data. *arXiv preprint arXiv:2602.04600*, 2026b.
- Jialong Li, Mingyue Zhang, Nianyu Li, Danny Weyns, Zhi Jin, and Kenji Tei. Generative ai for self-adaptive systems: State of the art and research roadmap. *ACM Transactions on Autonomous and Adaptive Systems*, 19(3):1–60, 2024.
- Xingzuo Li, Kehai Chen, Yunfei Long, Xuefeng Bai, Yong Xu, and Min Zhang. Generator-assistant stepwise rollback framework for large language model agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 17694–17711, 2025b.
- Xinzhe Li. A review of prominent paradigms for llm-based agents: Tool use, planning (including rag), and feedback learning. In *Proceedings of the 31st international conference on computational linguistics*, pp. 9760–9779, 2025.
- Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, et al. Memos: A memory os for ai system. *arXiv preprint arXiv:2507.03724*, 2025c.
- Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International conference on robotics and automation (ICRA)*, pp. 9493–9500. IEEE, 2023.
- Lizi Liao, Grace Hui Yang, and Chirag Shah. Proactive conversational agents in the post-chatgpt world. In *Proceedings of the 46th international ACM SIGIR conference on research and development in information retrieval*, pp. 3452–3455, 2023.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, et al. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026.
- Alisa Liu, Zhaofeng Wu, Julian Michael, Alane Suhr, Peter West, Alexander Koller, Swabha Swayamdipta, Noah A Smith, and Yejin Choi. We’re afraid language models aren’t modeling ambiguity. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pp. 790–807, 2023.
- Yibing Liu, Chong Zhang, Zhongyi Han, Hansong Liu, Yong Wang, Yang Yu, Xiaoyan Wang, and Yilong Yin. Trajad: Trajectory anomaly detection for trustworthy llm agents. *arXiv preprint arXiv:2602.06443*, 2026a.
- Zewen Liu, Zhan Shi, Yisi Sang, Bing He, Minhua Lin, Tianxin Wei, Dakuo Wang, Benoit Dumoulin, Wei Jin, and Hanqing Lu. Adaptive auto-harness: Sustained self-improvement for agentic system deployment on open-ended task streams. *arXiv preprint arXiv:2606.01770*, 2026b.
- Yiling Lou, Jun Yang, Samuel Benton, Dan Hao, Lin Tan, Zhenpeng Chen, Lu Zhang, and Lingming Zhang. When automated program repair meets regression testing—an extensive study on two million patches. *ACM Transactions on Software Engineering and Methodology*, 33(7):1–23, 2024.

-
- Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. Chameleon: Plug-and-play compositional reasoning with large language models. *ArXiv*, abs/2304.09842, 2023.
- Yaxi Lu, Shenzhi Yang, Cheng Qian, Guirong Chen, Qinyu Luo, Yesai Wu, Huadong Wang, Xin Cong, Zhong Zhang, Yankai Lin, et al. Proactive agent: Shifting llm agents from reactive responses to active assistance. In *International Conference on Learning Representations*, volume 2025, pp. 47431–47457, 2025.
- Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Skill0: In-context agentic reinforcement learning for skill internalization. *arXiv preprint arXiv:2604.02268*, 2026.
- Weidi Luo, Shenghong Dai, Xiaogeng Liu, Suman Banerjee, Huan Sun, Muhao Chen, and Chaowei Xiao. Agrail: A lifelong agent guardrail with effective and adaptive safety detection. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8104–8139, 2025.
- Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. Agentboard: An analytical evaluation board of multi-turn llm agents. *Advances in neural information processing systems*, 37:74325–74362, 2024.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- Pattie Maes. Agents that reduce work and information overload. *Communications of the ACM*, 37(7):30–40, 1994.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13851–13870, 2024.
- Arjun Majumdar, Anurag Ajay, Xiaohan Zhang, Pranav Putta, Sriram Yenamandra, Mikael Henaff, Sneha Silwal, Paul Mcvay, Oleksandr Maksymets, Sergio Arnaud, et al. Openeqa: Embodied question answering in the era of foundation models. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16488–16498. IEEE, 2024.
- Matheus Kunzler Maldaner, Adam Fourney, Amanda Swearngin, Hussein Mozannar, Gagan Bansal, Maya Murad, Rafah Hosn, and Saleema Amershi. Sentinelbench: A benchmark for long-running monitoring agents. *arXiv preprint arXiv:2606.05342*, 2026.
- Tula Masterman, Sandi Besen, Mason Sawtell, and Alex Chao. The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey. *arXiv preprint arXiv:2404.11584*, 2024.
- Michael McCloskey and Neal J Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of learning and motivation*, volume 24, pp. 109–165. Elsevier, 1989.
- Shuhaib Mehri, Priyanka Kargupta, Tal August, and Dilek Hakkani-Tür. Multisessioncollab: Learning user preferences with memory to improve long-term collaboration. *arXiv preprint arXiv:2601.02702*, 2026.
- Kai Mei, Xi Zhu, Wujiang Xu, Wenyue Hua, Mingyu Jin, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. Aios: Llm agent operating system. *arXiv preprint arXiv:2403.16971*, 2024.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.

-
- Johannes Moll, Jean-Philippe Corbeil, Jiazhen Pan, Martin Hadamitzky, Daniel Rueckert, Lisa Adams, and Keno Bressem. Grasp: Gated regression-aware skill proposer for self-improving llm agents. *arXiv preprint arXiv:2605.29668*, 2026.
- Yao Mu, Qinglong Zhang, Mengkang Hu, Wenhai Wang, Mingyu Ding, Jun Jin, Bin Wang, Jifeng Dai, Yu Qiao, and Ping Luo. Embodiedgpt: Vision-language pre-training via embodied chain of thought. *Advances in neural information processing systems*, 36:25081–25094, 2023.
- Hector Munoz-Avila, David W Aha, and Paola Rizzo. Chathtn: Interleaving approximate (llm) and symbolic htn planning. *arXiv preprint arXiv:2505.11814*, 2025.
- Robin R Murphy and Dave Hershberger. Handling sensing failures in autonomous mobile robots. *The International Journal of Robotics Research*, 18(4):382–400, 1999.
- Dang Nguyen, Viet Dac Lai, Seunghyun Yoon, Ryan A Rossi, Handong Zhao, Ruiyi Zhang, Puneet Mathur, Nedim Lipka, Yu Wang, Trung Bui, et al. Dynasaur: Large language agents beyond predefined actions. *arXiv preprint arXiv:2411.01747*, 2024.
- Dang Nguyen, Jian Chen, Yu Wang, Gang Wu, Namyong Park, Zhengmian Hu, Hanjia Lyu, Junda Wu, Ryan Aponte, Yu Xia, et al. Gui agents: A survey. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 22522–22538, 2025.
- Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. Semfix: Program repair via semantic analysis. In *2013 35th International Conference on Software Engineering (ICSE)*, pp. 772–781, 2013. doi: 10.1109/ICSE.2013.6606623.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjin Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026.
- Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*, 2019.
- Ann Nowé, Peter Vrancx, and Yann-Michaël De Hauwere. Game theory and multi-agent reinforcement learning. In *Reinforcement learning: State-of-the-art*, pp. 441–470. Springer, 2012.
- Besmira Nushi, Shamsi T Iqbal, Paul Bennett, Jaime Teevan, Eric Horvitz, Dan Weld, Ruth Kikin-Gil, Saleema Amershi, Penny Collisson, Mihaela Vorvoreanu, et al. Guidelines for human-ai interaction. In *Proceedings of the 2019 chi conference on human factors in computing systems*, 2019.
- John Mark Ockerbloom and Robert J Allen. Architectural mismatch: Why reuse is so hard. *IEEE software*, 12(6):17–26, 1995.
- Kai Tzu-iunn Ong, Namyoun Kim, Minju Gwak, Hyungjoo Chae, Taeyoon Kwon, Yohan Jo, Seung-won Hwang, Dongha Lee, and Jinyoung Yeo. Towards lifelong dialogue agents via timeline-based memory management. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 8631–8661, 2025.
- Peyman Oreizy, Nenad Medvidovic, and Richard N Taylor. Architecture-based runtime software evolution. In *Proceedings of the 20th international conference on Software engineering*, pp. 177–186. IEEE, 1998.
- Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations*, volume 2026, pp. 94327–94354, 2026.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

-
- Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-language agent harnesses. *arXiv preprint arXiv:2603.25723*, 2026a.
- Wenbo Pan, Shujie Liu, Chin-Yew Lin, Jingying Zeng, Xianfeng Tang, Xiangyang Zhou, Yan Lu, and Xiaohua Jia. Retrospective harness optimization: Improving llm agents via self-preference over trajectory rollouts. *arXiv preprint arXiv:2606.05922*, 2026b.
- German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural networks*, 113:54–71, 2019.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pp. 1–22, 2023.
- David Lorge Parnas. Information distribution aspects of design methodology. 1971.
- David Lorge Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.
- Gil Pasternak, Dheeraj Rajagopal, Julia White, Dhruv Atreja, Matthew Thomas, George Hurn-Maloney, and Ash Lewis. Beyond reactivity: Measuring proactive problem solving in llm agents. *arXiv preprint arXiv:2510.19771*, 2025.
- Vedant Patel. Supersede: Diagnosing and training the memory-update gap in llm agents. *arXiv preprint arXiv:2606.27472*, 2026.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024.
- David Patterson, Aaron Brown, Pete Broadwell, George Candea, Mike Chen, James Cutler, Patricia Enriquez, Armando Fox, Emre Kiciman, Matthew Merzbacher, et al. Recovery-oriented computing (roc): Motivation, definition, techniques, and case studies. Technical report, Technical Report UCB//CSD-02-1175, UC Berkeley Computer Science, 2002.
- Dewayne E Perry and Alexander L Wolf. Foundations for the study of software architecture. *ACM SIGSOFT Software engineering notes*, 17(4):40–52, 1992.
- Arne Peters and Alois C Knoll. Robot self-calibration using actuated 3d sensors. *Journal of Field Robotics*, 41(2):327–346, 2024.
- Kevin Pu, Ting Zhang, Naveen Sendhilnathan, Sebastian Freitag, Raj Sodhi, and Tanya R Jonker. Promemasist: Exploring timely proactive assistance through working memory modeling in multi-modal wearable devices. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–19, 2025.
- Cheng Qian, Chi Han, Yi Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. Creator: Tool creation for disentangling abstract and concrete reasoning of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 6922–6939, 2023.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, volume 2024, pp. 9695–9717, 2024.
- Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.
- Anand S Rao, Michael P Georgeff, et al. Bdi agents: From theory to practice. In *Icmas*, volume 95, pp. 312–319, 1995.

-
- Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pp. 2001–2010, 2017.
- Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, et al. A generalist agent. *arXiv preprint arXiv:2205.06175*, 2022.
- Allen Z Ren, Jaden Clark, Anushri Dixit, Masha Itkina, Anirudha Majumdar, and Dorsa Sadigh. Explore until confident: Efficient exploration for embodied question answering. *arXiv preprint arXiv:2403.15941*, 2024.
- Zhe Ren, Yimeng Chen, Dandan Guo, Guowei Rong, Tonghui Li, RB Xiong, Qingfeng Lan, Wenyi Wang, Li Nanbo, Yibo Yang, et al. Self-improvements in modern agentic systems: A survey. *arXiv preprint arXiv:2607.13104*, 2026.
- Alireza Rezazadeh, Zichao Li, Ange Lou, Yuying Zhao, Wei Wei, and Yujia Bao. Collaborative memory: Multi-user memory sharing in llm agents with dynamic access control. *arXiv preprint arXiv:2505.18279*, 2025.
- Bradley J Rhodes and Pattie Maes. Just-in-time information retrieval agents. *IBM Systems journal*, 39(3.4): 685–704, 2000.
- Dmitriy Rivkin, Francois Hogan, Amal Feriani, Abhisek Konar, Adam Sigal, Steve Liu, and Greg Dudek. Sage: Smart home agent with grounded execution. *arXiv preprint arXiv:2311.00772*, 2023.
- Dmitriy Rivkin, Francois Hogan, Amal Feriani, Abhisek Konar, Adam Sigal, Xue Liu, and Gregory Dudek. Aiot smart home via autonomous llm agents. *IEEE Internet of Things Journal*, 12(3):2458–2472, 2024.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and trends® in information retrieval*, 4(1-2):1–174, 2009.
- Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.
- Pascal J Sager, Benjamin Meyer, Peng Yan, Rebekka von Wartburg-Kottler, Layan Etaiwi, Aref Enayati, Gabriel Nobel, Ahmed Abdulkadir, Benjamin F Grewe, and Thilo Stadelmann. A comprehensive survey of agents for computer use: Foundations, challenges, and future directions. *Journal of Artificial Intelligence Research*, 85, 2026.
- Igor Santos-Grueiro. Temporary authority, permanent effects: Commit-time authorization for llm agents. *arXiv preprint arXiv:2607.10487*, 2026.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. Agentsquare: Automatic llm agent search in modular design space. In *International Conference on Learning Representations*, volume 2025, pp. 3841–3865, 2025.
- Shuai Shao, Qihan Ren, Dongrui Liu, Chen Qian, Boyi Wei, Dadi Guo, Jingyi Yang, Xinhao Song, Linfeng Zhang, Weinan Zhang, et al. Your agent may misevolve: Emergent risks in self-evolving llm agents. In *International Conference on Learning Representations*, volume 2026, pp. 99728–99793, 2026a.
- Yijia Shao, Vinay Samuel, Yucheng Jiang, John Yang, and Diyi Yang. Collaborative gym: A framework for enabling and evaluating human-agent collaboration. In *International Conference on Learning Representations*, volume 2026, pp. 99616–99649, 2026b.

-
- Yiting Shen, Kun Li, Wei Zhou, and Songlin Hu. Mem2actbench: A benchmark for evaluating long-term memory utilization in task-oriented autonomous agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8173–8190, 2026.
- Haizhou Shi, Zihao Xu, Hengyi Wang, Weiyi Qin, Wenyuan Wang, Yibin Wang, Zifeng Wang, Sayna Ebrahimi, and Hao Wang. Continual learning of large language models: A comprehensive survey. *ACM Computing Surveys*, 58(5):1–42, 2025.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- Yoav Shoham. Agent-oriented programming. *Artificial intelligence*, 60(1):51–92, 1993.
- Shuzheng Si, Haozhe Zhao, Yu Lei, Qingyi Wang, Dingwei Chen, Zhitong Wang, Zhenhailong Wang, Kangyang Luo, Zheng Wang, Gang Chen, et al. From context to skills: Can language models learn from context skillfully? *arXiv preprint arXiv:2604.27660*, 2026.
- Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv:2209.11302*, 2022.
- Nishad Singhi, Christian Bialas, Snehal Jauhri, Vignesh Prasad, Georgia Chalvatzaki, Marcus Rohrbach, and Anna Rohrbach. Think twice, act once: Verifier-guided action selection for embodied agents. *arXiv preprint arXiv:2605.12620*, 2026.
- Marta Skreta, Zihan Zhou, Jia Lin Yuan, Kourosh Darvish, Alán Aspuru-Guzik, and Animesh Garg. Replan: Robotic replanning with perception and language models. *arXiv preprint arXiv:2401.04157*, 2024.
- Chan Hee Song, Jiaman Wu, Clay Washington, Brian M. Sadler, Wei-Lun Chao, and Yu Su. Llm-planner: Few-shot grounded planning for embodied agents with large language models. *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 2986–2997, 2022.
- Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, and Alex Pentland. Authenticated delegation and authorized ai agents. *arXiv preprint arXiv:2501.09674*, 2025.
- Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, et al. Paperbench: Evaluating ai’s ability to replicate ai research. *arXiv preprint arXiv:2504.01848*, 2025.
- Miao Su, Yucan Guo, Zhongni Hou, Long Bai, Zixuan Li, Yufei Zhang, Guojun Yin, Wei Lin, Xiaolong Jin, Jiafeng Guo, et al. Beyond dialogue time: Temporal semantic memory for personalized llm agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 29935–29951, 2026.
- Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*, 2023.
- Haoran Sun, Zekun Zhang, and Shaoning Zeng. Preference-aware memory update for long-term llm agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 783–793, 2026a.
- Zeyi Sun, Ziyu Liu, Yuhang Zang, Yuhang Cao, Xiaoyi Dong, Tong Wu, Dahua Lin, and Jiaqi Wang. Seagent: Self-evolving computer use agent with autonomous learning from experience. *arXiv preprint arXiv:2508.04700*, 2025.
- Zhaoyan Sun, Shan Zhong, Daizhou Wen, Jiaxing Han, Guoliang Li, Ying Yan, Peng Zhang, Yu Su, Xiang Qi, Baolin Sun, Chengyuan Yang, Tao Fang, and Huaiyu Ruan. Agenticdatabench: A comprehensive benchmark for data agents, 2026b. URL <https://arxiv.org/abs/2607.01647>.

-
- D’idac Sur’is, Sachit Menon, and Carl Vondrick. Vipergpt: Visual inference via python execution for reasoning. *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 11854–11864, 2023.
- Zhen Tan, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long Le, Yiwen Song, Yanfei Chen, Hamid Palangi, George Lee, et al. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8416–8439, 2025.
- Yuanbo Tang, Huaze Tang, Tingyu Cao, Lam Nguyen, Anping Zhang, Xinwen Cao, Chunkang Liu, Wenbo Ding, and Yang Li. Proagentbench: Evaluating llm agents for proactive assistance with real-world data. *arXiv preprint arXiv:2602.04482*, 2026a.
- Zirui Tang, Xuanhe Zhou, Yumou Liu, Linchun Li, Yukai Wu, Weizheng Wang, Hongzhang Huang, Wei Zhou, Jun Zhou, Jiachen Song, Shaoli Yu, Jinqi Wang, Zihang Zhou, Hongyi Zhou, Yuting Lv, Jinyang Li, Jiashuo Liu, Ruoyu Chen, Chunwei Liu, Guoliang Li, Jihua Kang, and Fan Wu. Workspace-bench 1.0: Benchmarking ai agents on workspace tasks with large-scale file dependencies, 2026b. URL <https://arxiv.org/abs/2605.03596>.
- Keda Tao, Wenjie Du, Bohan Yu, Weiqiang Wang, Jian Liu, and Huan Wang. Active perception agent for omnimodal audio-video understanding. *arXiv preprint arXiv:2512.23646*, 2025.
- Stefanie Tellex, Ross A. Knepper, Adrian Shuai Li, Daniela Rus, and Nicholas Roy. Asking for help using inverse semantics. In *Robotics: Science and Systems Conference*, 2014.
- Bo Wang, Yuqian Yao, Enxi Wang, Luozhijie Jin, Yang Liu, Yiran Suo, Yuxuan Cai, Enyu Zhou, Yufei Gao, Honglin Guo, Tianyu Huai, Li Ji, Zhikai Lei, Bufan Li, Lizhi Lin, Jinxiu Liu, Jie Yang, Jiazheng Zhou, Maosen Zhou, Pengfang Qian, Shichun Liu, Guanshan Liu, Hao Zheng, Yunhao Yu, Hang Yan, Jihua Kang, Xinchu Chen, and Xipeng Qiu. Contextweave: A real-world workflow benchmark, 2026a. URL <https://arxiv.org/abs/2608.04830>.
- Bowen Wang, Xinyuan Wang, Jiaqi Deng, Tianbao Xie, Ryan Li, Yanzhe Zhang, Junli Wang, Dunjie Lu, Zicheng Gong, Gavin Li, et al. Computer agent arena: Toward human-centric evaluation and analysis of computer-use agents. In *International Conference on Learning Representations*, volume 2026, pp. 58008–58068, 2026b.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023a.
- Haoyu Wang, Christopher M Poskitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents. *arXiv preprint arXiv:2503.18666*, 2025a.
- Haoyu Wang, Christopher M Poskitt, Jiali Wei, and Jun Sun. Proguard: Probabilistic runtime monitoring for llm agent safety. *arXiv preprint arXiv:2508.00500*, 2025b.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of computer science*, 18(6):186345, 2024a.
- Shuoxin Wang, Chang Liu, Gowen Loo, Lifan Zheng, Kaiwen Wei, Huanqian Yan, Xinyi Zeng, Jingyuan Zhang, and Yu Tian. Me-agent: A personalized mobile agent with two-level user habit learning for enhanced interaction. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 24206–24222, 2026c.
- Wenxuan Wang, Shi Juluan, Zixuan Ling, Yuk-Kit Chan, Chaozheng Wang, Cheryl Lee, Youliang Yuan, Jen-tse Huang, Wenxiang Jiao, and Michael R Lyu. Learning to ask: When llm agents meet unclear instruction. In *Proceedings of the 2025 conference on empirical methods in natural language processing*, pp. 21784–21795, 2025c.

-
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. *ArXiv*, abs/2402.01030, 2024b.
- Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Wu, et al. Opencua: Open foundations for computer-use agents. *Advances in Neural Information Processing Systems*, 38:139756–139806, 2026d.
- Yuanfei Wang, Xinju Huang, Fangwei Zhong, Yaodong Yang, Yizhou Wang, Yuanpei Chen, and Hao Dong. From strangers to assistants: Fast desire alignment for embodied agent-user adaptation. *arXiv e-prints*, pp. arXiv-2505, 2025d.
- Zhihao Wang, Jianxiong Li, Jinliang Zheng, Wencong Zhang, Dongxiu Liu, Yinan Zheng, Haoyi Niu, Junzhi Yu, and Xianyuan Zhan. Physiagent: An embodied agent framework in physical world. *arXiv preprint arXiv:2509.24524*, 2025e.
- Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojuan Ma, and Yitao Liang. Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents. *arxiv 2023. arXiv preprint arXiv:2302.01560*, 10, 2023b.
- Ziyang Wang, Honglu Zhou, Shijie Wang, Junnan Li, Caiming Xiong, Silvio Savarese, Mohit Bansal, Michael S Ryoo, and Juan Carlos Niebles. Active video perception: Iterative evidence seeking for agentic long video understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9088–9099, 2026e.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024c.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Lai Wei, Chengqi Li, Jiapeng Li, Ruina Hu, Yue Wang, and Weiran Huang. Clbench-v: Evaluating multimodal context learning from grounding to knowledge acquisition. *arXiv preprint arXiv:2607.25294*, 2026.
- Xinming Wei, Jiahao Zhang, Haoran Li, Jiayu Chen, Haoning Guan, Rui Qu, Maoliang Li, Xiang Chen, and Guojie Luo. Agent. xpu: Efficient scheduling of agentic llm workloads on heterogeneous soc. *arXiv preprint arXiv:2506.24045*, 2025.
- Michael Wooldridge and Nicholas R Jennings. Intelligent agents: Theory and practice. *The knowledge engineering review*, 10(2):115–152, 1995.
- Bin Wu, Guanyun Zou, Bingbing Wang, Huan Zhao, and Chuan Shi. Ask now, use later: Benchmarking the proactivity gap in long-lived llm agents. *arXiv preprint arXiv:2605.28108*, 2026a.
- Chenfei Wu, Sheng-Kai Yin, Weizhen Qi, Xiaodong Wang, Zecheng Tang, and Nan Duan. Visual chatgpt: Talking, drawing and editing with visual foundation models. *ArXiv*, abs/2303.04671, 2023a.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024a.
- Qianhui Wu, Kanzhi Cheng, Rui Yang, Chaoyun Zhang, Jianwei Yang, Huiqiang Jiang, Jian Mu, Baolin Peng, Bo Qiao, Reuben Tan, et al. Gui-actor: Coordinate-free visual grounding for gui agents. *Advances in Neural Information Processing Systems*, 38:15101–15128, 2026b.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023b.

-
- Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. Os-copilot: Towards generalist computer agents with self-improvement. *arXiv preprint arXiv:2402.07456*, 2024b.
- Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *Science China information sciences*, 68(2):121101, 2025.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.
- Zhifei Xie, Zongzheng Hu, Fangda Ye, Xin Zhang, Haobo Chai, Zihang Liu, Pengcheng Wu, Guibin Zhang, Yue Liao, Xiaobin Hu, et al. Pask: Toward intent-aware proactive agents with long-term memory. *arXiv preprint arXiv:2604.08000*, 2026.
- Zhiqiang Xie, Hao Kang, Ying Sheng, Tushar Krishna, Kayvon Fatahalian, and Christos Kozyrakis. Ai metropolis: Scaling large language model-based multi-agent simulation with out-of-order execution. *Proceedings of Machine Learning and Systems*, 7, 2025.
- Frank Fangzheng Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, et al. Theagentcompany: benchmarking llm agents on consequential real world tasks. *Advances in Neural Information Processing Systems*, 38, 2026a.
- Wei Xu, Ling Huang, Armando Fox, David Patterson, and Michael I. Jordan. Detecting large-scale system problems by mining console logs. ICML’10, pp. 37–46, Madison, WI, USA, 2010. Omnipress. ISBN 9781605589077.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38:17577–17604, 2026b.
- Yue Xu, Qian Chen, Zizhan Ma, Dongrui Liu, Wenxuan Wang, Xiting Wang, Li Xiong, and Wenjie Wang. Toward personalized llm-powered agents: Foundations, evaluation, and future directions. *arXiv preprint arXiv:2602.22680*, 2026c.
- Yunhe Yan, Shihe Wang, Jiajun Du, Yexuan Yang, Yuxuan Shan, Qichen Qiu, Xianqing Jia, Xinge Wang, Xin Yuan, Xu Han, et al. Mcpworld: A unified benchmarking testbed for api, gui, and hybrid computer use agents. *arXiv preprint arXiv:2506.07672*, 2025.
- Bufang Yang, Yunqi Guo, Lilin Xu, Zhenyu Yan, Hongkai Chen, Guoliang Xing, and Xiaofan Jiang. Socialmind: Llm-based proactive ar social assistive system with human-like perception for in-situ live interactions. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 9(1):1–30, 2025a.
- Bufang Yang, Lilin Xu, Liekang Zeng, Yunqi Guo, Siyang Jiang, Wenrui Lu, Kaiwei Liu, Yixuan Li, Xiaofan Jiang, Guoliang Xing, et al. Proagent: Harnessing on-demand sensory contexts for proactive llm agent systems in the wild. *arXiv preprint arXiv:2512.06721*, 2025b.
- Bufang Yang, Lilin Xu, Liekang Zeng, Kaiwei Liu, Siyang Jiang, Wenrui Lu, Hongkai Chen, Xiaofan Jiang, Guoliang Xing, and Zhenyu Yan. Contextagent: Context-aware proactive llm agents with open-world sensory perceptions. *Advances in Neural Information Processing Systems*, 38:167509–167543, 2025c.
- Chengyuan Yang, Zequn Sun, Wei Wei, and Wei Hu. Beyond static summarization: Proactive memory extraction for llm agents. *arXiv preprint arXiv:2601.04463*, 2026a.
- John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. Intercode: standardizing and benchmarking interactive coding with execution feedback. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS ’23, Red Hook, NY, USA, 2023. Curran Associates Inc.

-
- Shuo Yang, Caren Han, Xueqi Ma, Yan Li, Mohammad Reza Ghasemi Madani, and Eduard Hovy. Evotool: Self-evolving tool-use policy optimization in llm agents via blame-aware mutation and diversity-aware selection. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 43553–43572, 2026b.
- Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, et al. Skillopt: Executive strategy for self-evolving agent skills. *arXiv preprint arXiv:2605.23904*, 2026c.
- Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, et al. Autoskill: Experience-driven lifelong learning via skill self-evolution. *arXiv preprint arXiv:2603.01145*, 2026d.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35: 20744–20757, 2022a.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *NeurIPS 2022 Foundation Models for Decision Making Workshop*, 2022b.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 27890–27913, 2025.
- Hyungjun Yoon, Mohammad Malekzadeh, Sung-Ju Lee, Fahim Kawsar, and Lorena Qendro. Consensus: Multi-agent collaboration for multimodal sensing. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 32950–32969, 2026.
- Junwei Yu, Yepeng Ding, and Hiroyuki Sato. Dyntaskmas: A dynamic task graph-driven framework for asynchronous and parallel llm-based multi-agent systems. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 35, pp. 288–296, 2025a.
- Xiaofan Yu, Lanxiang Hu, Benjamin Reichman, Dylan Chu, Rushil Chandrupatla, Xiyuan Zhang, Larry Heck, and Tajana S Rosing. Sensorchat: Answering qualitative and quantitative questions during long-term multimodal sensor interactions. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 9(3):1–35, 2025b.
- Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 21457–21483, 2026.
- Youssef Zaky, Gaurav Paruthi, Bryan Tripp, and James Bergstra. Active perception and representation for robotic manipulation. *arXiv preprint arXiv:2003.06734*, 2020.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation. *arXiv preprint arXiv:2310.02304*, 2023.
- Andy Zeng, Maria Attarian, Brian Ichter, Krzysztof Choromanski, Adrian Wong, Stefan Welker, Federico Tombari, Aveek Purohit, Michael Ryoo, Vikas Sindhwani, et al. Socratic models: Composing zero-shot multimodal reasoning with language. *arXiv preprint arXiv:2204.00598*, 2022.
- Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. Agenttuning: Enabling generalized agent abilities for llms. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3053–3077, 2024.

-
- Ceyao Zhang, Kaijie Yang, Siyi Hu, Zihao Wang, Guanghe Li, Yihang Sun, Cheng Zhang, Zhaowei Zhang, Anji Liu, Song-Chun Zhu, et al. Proagent: building proactive cooperative agents with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 17591–17599, 2024a.
- Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. Appagent: Multimodal agents as smartphone users. In *Proceedings of the 2025 CHI conference on human factors in computing systems*, pp. 1–20, 2025a.
- Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026a.
- Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. Memskill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*, 2026b.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: open-ended evolution of self-improving agents. In *International Conference on Learning Representations*, volume 2026, pp. 104223–104294, 2026c.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. Aflow: Automating agentic workflow generation. In *International Conference on Learning Representations*, volume 2025, pp. 34040–34077, 2025b.
- Wenqi Zhang, Ke Tang, Hai Wu, Mengna Wang, Yongliang Shen, Guiyang Hou, Zeqi Tan, Peng Li, Yueting Zhuang, and Weiming Lu. Agent-pro: Learning to evolve via policy-level reflection and optimization. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5348–5375, 2024b.
- Wenyuan Zhang, Xinghua Zhang, Haiyang Yu, Shuaiyi Nie, Bingli Wu, Juwei Yue, Tingwen Liu, and Yongbin Li. Expseek: Self-triggered experience seeking for web agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 29255–29283, 2026d.
- Xuan Zhang, Yang Deng, Zifeng Ren, See Kiong Ng, and Tat-Seng Chua. Ask-before-plan: Proactive language agents for real-world planning. In *Findings of the association for computational linguistics: EMNLP 2024*, pp. 10836–10863, 2024c.
- Yaolun Zhang, Yiran Wu, Yijiong Yu, Qingyun Wu, and Huazheng Wang. Live-evo: Online evolution of agentic memory from continuous feedback. *arXiv preprint arXiv:2602.02369*, 2026e.
- Yinger Zhang, Shutong Jiang, Renhao Li, Jianhong Tu, Yang Su, Lianghao Deng, Xudong Guo, Chenxu Lv, and Junyang Lin. Deepplanning: Benchmarking long-horizon agentic planning with verifiable constraints. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7377–7407, 2026f.
- Yingqi Zhang. Agent libos: A runtime substrate for capability-controlled self-evolving llm agents. *arXiv preprint arXiv:2606.03895*, 2026.
- Yuwei Zhang, Kumar Ayush, Siyuan Qiao, A Ali Heydari, Girish Narayanswamy, Max Xu, Ahmed Metwally, Jinhua Xu, Jake Garrison, Xuhai Xu, et al. Sensorlm: Learning the language of wearable sensors. *Advances in Neural Information Processing Systems*, 38:46813–46849, 2026g.
- Yuzhe Zhang, Xianwei Xue, Xingyong Wu, Mengke Chen, Chen Liu, Xinran He, Run Shao, Feiran Liu, Huanmin Xu, Qiutong Pan, et al. Don’t act blindly: Robust gui automation via action-effect verification and self-correction. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 28924–28941, 2026h.
- Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6):1–47, 2025c.

-
- Zhaoxi Zhang and Xiaomei Zhang. Are you still the agent i authorized? earned authority under a fixed ceiling for evolving agents. *arXiv preprint arXiv:2607.23586*, 2026.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19632–19642, 2024.
- Yuheng Zhao, Xueli Shu, Liwen Fan, Lin Gao, Yu Zhang, and Siming Chen. Proactiveva: Proactive visual analytics with llm-based ui agent. *IEEE Transactions on Visualization and Computer Graphics*, 2025.
- Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, ZhongZhi Li, Yingying Zhang, Le Song, and Qianli Ma. Lifelongagentbench: Evaluating llm agents as lifelong learners. *arXiv preprint arXiv:2505.11942*, 2025a.
- Junhao Zheng, Shengjie Qiu, Chengming Shi, and Qianli Ma. Towards lifelong learning of large language models: A survey. *ACM Computing Surveys*, 57(8):1–35, 2025b.
- Junhao Zheng, Chengming Shi, Xidi Cai, Qiuke Li, Duzhen Zhang, Chenxing Li, Dong Yu, and Qianli Ma. Lifelong learning of large language model based agents: A roadmap. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026a.
- Yusheng Zheng, Jiakun Fan, Quanzhi Fu, Yiwei Yang, Wei Zhang, and Andi Quinn. Agentcgroup: Understanding and controlling os resources of ai agents. *arXiv preprint arXiv:2602.09345*, 2026b.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pp. 19724–19731, 2024.
- Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*, 2023.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, volume 2024, pp. 15585–15606, 2024.
- Jianing Zhu, Yeonju Ro, John Robertson, Kevin Wang, Junbo Li, Haris Vikalo, Aditya Akella, and Zhangyang Wang. Your agents are aging too: Agent lifespan engineering for deployed systems. *arXiv preprint arXiv:2605.26302*, 2026.
- Jinhao Zhu, Kevin Tseng, Gil Vernik, Xiao Huang, Shishir G Patil, Vivian Fang, and Raluca Ada Popa. Miniscope: A least privilege framework for authorizing tool calling agents. *arXiv preprint arXiv:2512.11147*, 2025.
- Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Language agents as optimizable graphs. In *International Conference on Machine Learning*, 2024a.
- Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, et al. Agent-as-a-judge: Evaluate agents with agents. *arXiv preprint arXiv:2410.10934*, 2024b.
- Pouyan Ziafati, Mehdi Dastani, John-Jules Meyer, and Leon van der Torre. Event-processing in autonomous robot programming. In *AAMAS. International Foundation for Autonomous Agents and Multiagent Systems*, 2013.